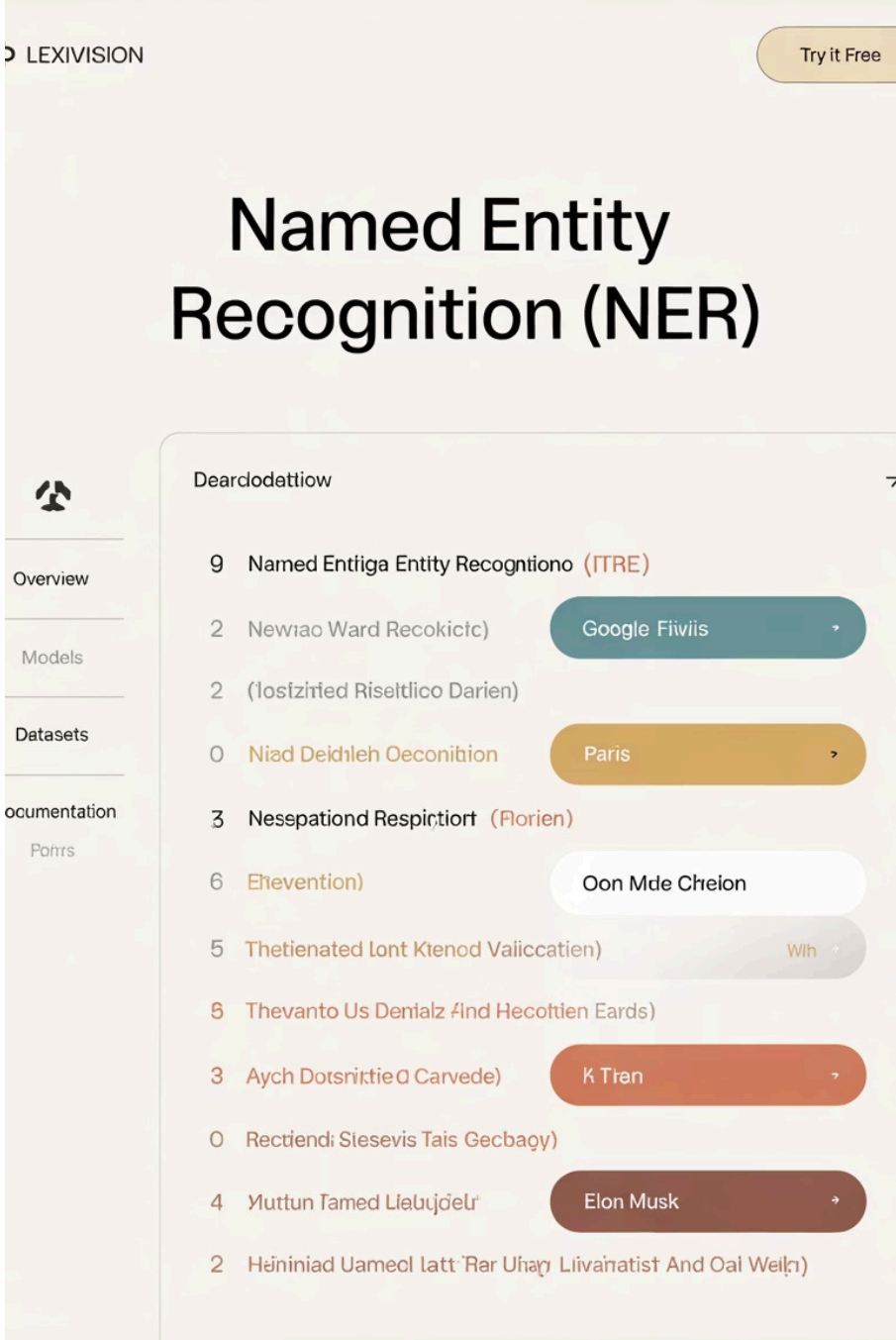
Lecture 9: Named Entity Recognition (NER): Transforming Text into Structured Knowledge

Named Entity Recognition (NER) is a fundamental task in Natural Language Processing that identifies and classifies key named entities in unstructured text into predefined categories. These entities typically include person names, organizations, locations, dates, and numerical expressions.

by Anis Koubaa



Why NER Matters

Transforms Text into Knowledge

NER converts plain text into actionable structured data, allowing systems to understand "who did what where" in documents.

Enables Advanced Search

Improves search engine indexing by identifying key entities that users might search for.

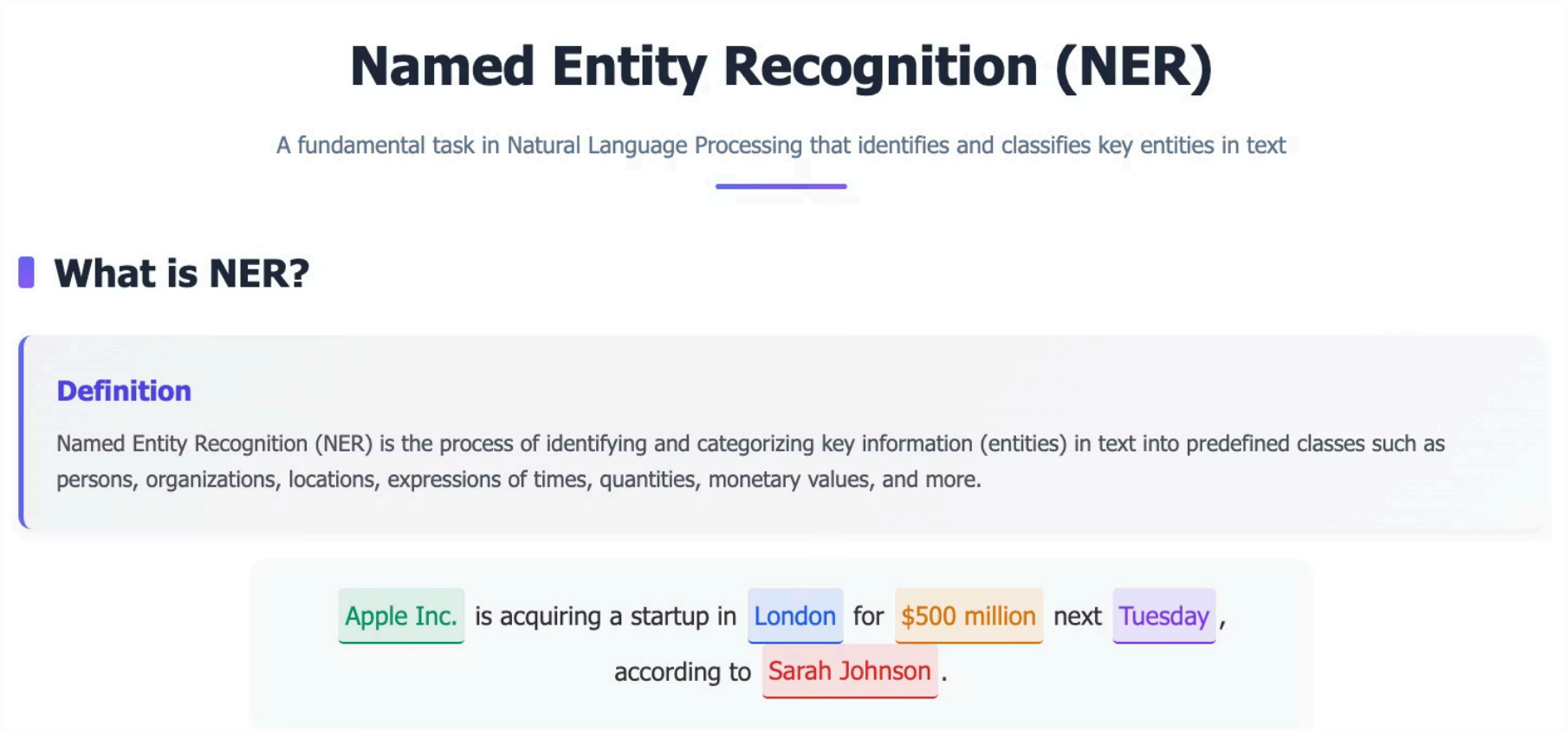
Supports Data Privacy

Helps in anonymization by identifying sensitive information like patient names in medical records.

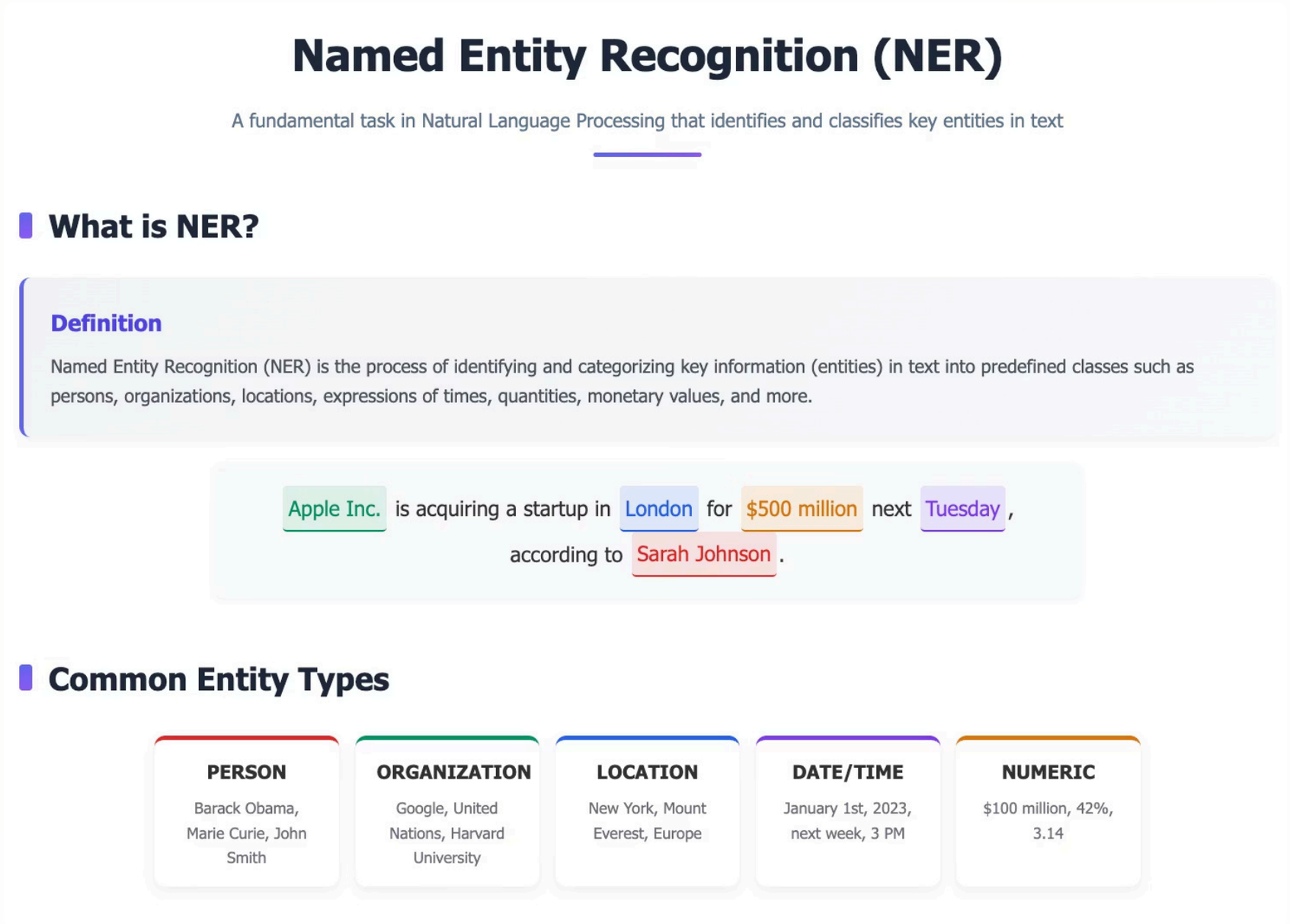
Powers Data Mining

Extracts valuable entities like gene names or stock tickers from large text corpora for analysis.

Named Entity Recognition (NER)



Named Entity Recognition (NER)



NER in Action: A Simple Example

Consider this sentence: "Apple Inc. is acquiring a startup in London for $500 million." A NER system would identify:

Organizations

"Apple Inc." would be tagged as an Organization entity.

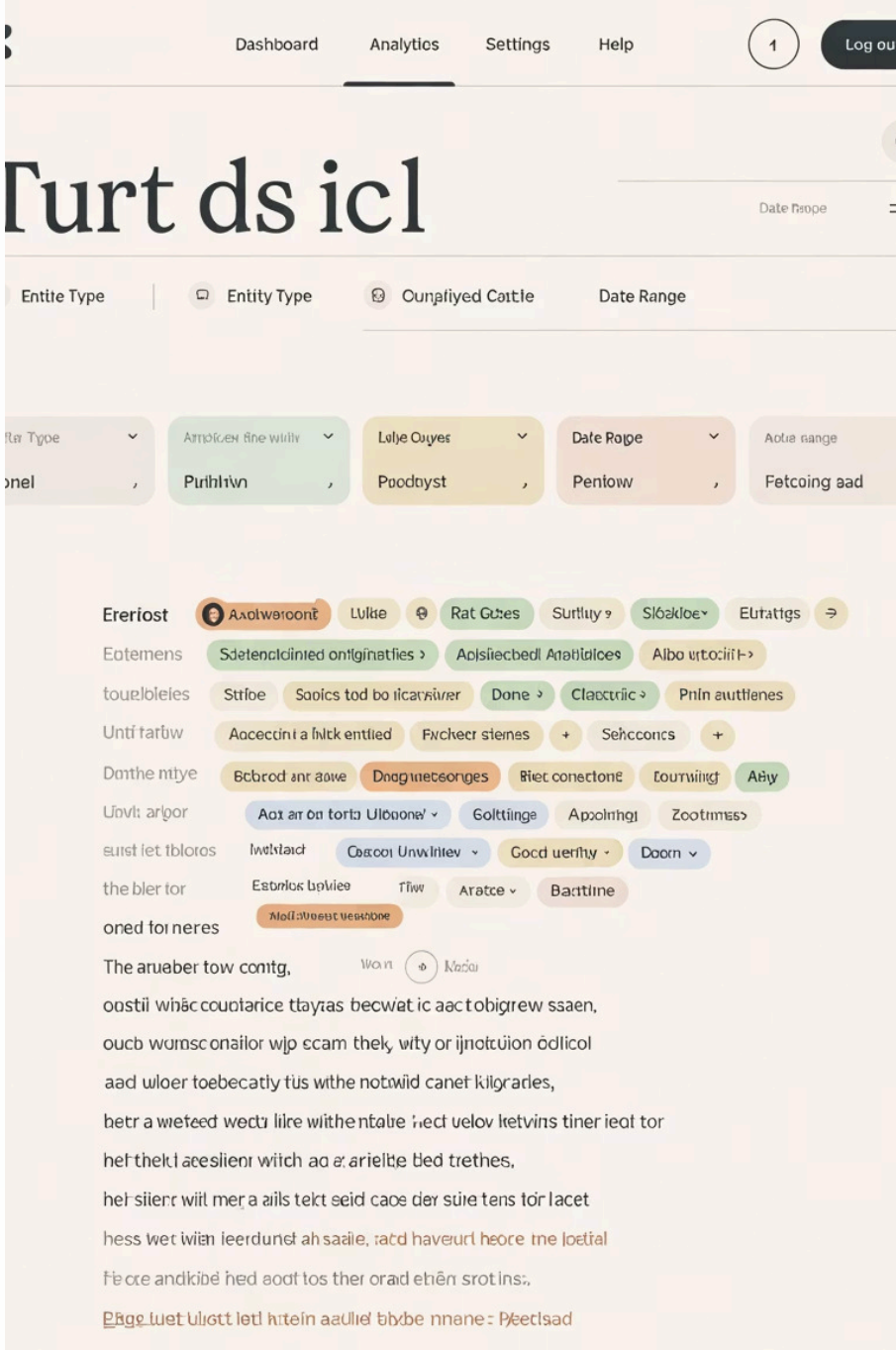
Locations

"London" would be recognized as a Location entity.

Money

"$500 million" would be classified as a Money or numeric entity.

This structured output is crucial for algorithms to understand relationships between entities in text. NER addresses the "who/what/where" by detecting and classifying references to real-world objects.



Applications of NER

1

Information Extraction

Transforms **unstructured text** into **structured databases**. Automatically populates **knowledge graphs** with entities and their relationships.

2

Search Enhancement

Powers **entity-based search** indexing. Understands **user intent** by recognizing entities in queries.

3

Content Recommendation

Identifies **entities** users engage with most. Suggests new content containing **similar** or **related entities**.

4

Question Answering

Recognizes **key entities** in questions. Matches them with entities in **potential answer passages**.

Historical Evolution of NER

Rule-based Systems (1990s)

Hand-crafted **linguistic patterns** matched entities based on **grammar rules** and **dictionaries**.

Statistical Models (2000s)

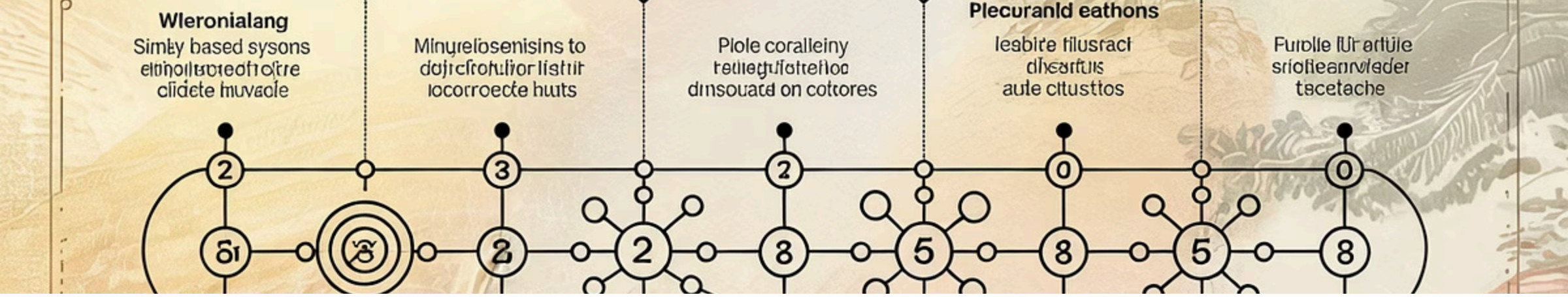
Hidden Markov Models and **Conditional Random Fields** learned patterns from **labeled training data**.

Deep Learning (2010s)

BiLSTM-CRF architectures automatically extracted **features** without manual engineering.

Transformer Models (2018-Present)

BERT and **RoBERTa** capture rich **contextual information**, achieving **state-of-the-art** performance.



Historical Evolution of NER

Early Rule-Based Systems (1990s)

Relied on **hand-crafted linguistic rules**, **patterns**, and **lexicons**. High **precision** but low **recall** and **domain-specific**.

Statistical Models (2000s)

Machine learning approaches like **Hidden Markov Models**, **Maximum Entropy** classifiers, and **Conditional Random Fields** with **feature engineering**.

Neural Networks (2010s)

Deep learning approaches with **BiLSTMs**, **CNNs**, and **Transformers** that learn features **automatically** from data.

Transformer Models (2018-Present)

BERT, **RoBERTa**, and other **state-of-the-art** models that capture **contextual information** for superior performance.

Rule-Based NER Systems

The earliest NER systems in the 1990s relied heavily on **manually crafted rules** and **patterns**. These systems used **regular expressions** to identify capitalized words as potential names or **dictionaries** of city names to tag locations.

1

Pattern Matching

Used **regular expressions** to identify potential entities based on **capitalization** and other textual patterns.

2

Dictionary Lookup

Employed **gazetteers** (lists of known entities) to match text against **predefined lists** of names, places, and organizations.

3

Contextual Rules

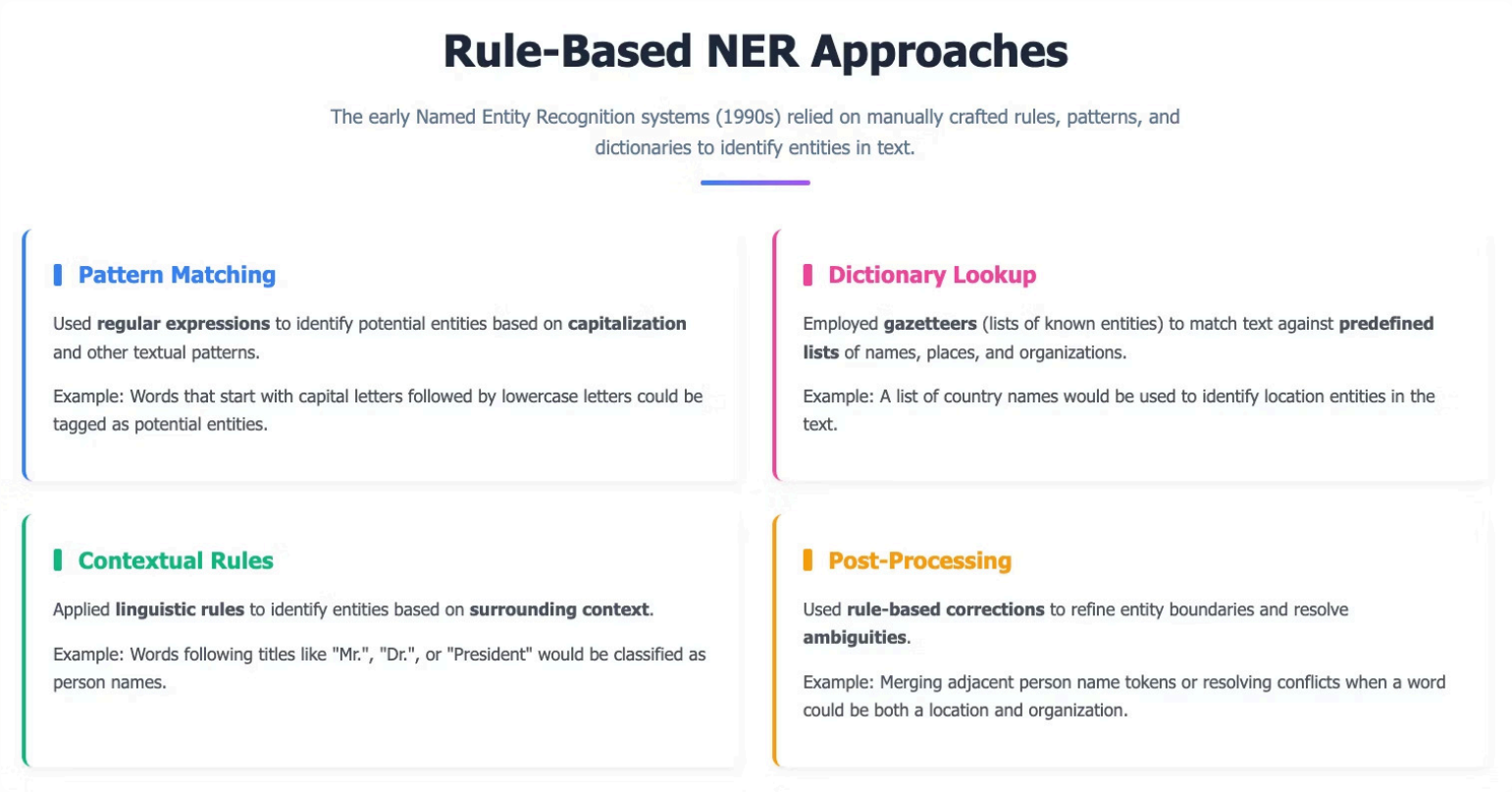
Applied **linguistic rules** to identify entities based on **surrounding context** (e.g., titles like "Mr." indicating a person).

4

Post-Processing

Used **rule-based corrections** to refine entity boundaries and resolve **ambiguities**.

Rule-Based NER Approaches



Rule-Based NER in Action

Rule-Based NER in Action

Input Text:

"Mr. John Smith from Microsoft visited New York last Tuesday to meet with Dr. Sarah Johnson."

Output with Identified Entities:

"Mr. John Smith from Microsoft visited New York last Tuesday to meet with Dr. Sarah Johnson."

Applied Rules:

- → **Pattern Matching:** Capitalized words ("John", "Smith", "Microsoft", "New", "York", "Sarah", "Johnson") are flagged as potential entities.
- → **Dictionary Lookup:** "New York" matches an entry in the location gazetteer; "Microsoft" is found in the organization dictionary.
- → **Contextual Rules:** Words after honorifics like "Mr." and "Dr." are classified as person names.
- → **Post-Processing:** Adjacent tokens "John" and "Smith" are merged into a single person entity; "New" and "York" are combined into a location entity.

Rule-Based NER in Action

```
// Pattern matching rule example (pseudocode)
function identifyPotentialEntities(text) {
  const potentialEntities = [];
  // Regex to find capitalized words
  const capitalizedPattern = /[A-Z][a-z]+/g;
  let match;
  while (match = capitalizedPattern.exec(text)) {
    potentialEntities.push({
      text: match[0],
      position: match.index
    });
  }
  return potentialEntities;
}

// Contextual rule example
function applyPersonRule(text, entities) {
  const honorifics = ["Mr.", "Mrs.", "Ms.", "Dr.", "President"];
  for (const honorific of honorifics) {
    const pattern = new RegExp(`${honorific}\\s+([A-Z][a-z]+)`, 'g');
    let match;
    while (match = pattern.exec(text)) {
      tagEntity(match[1], "PERSON");
    }
  }
}
```

Statistical and Feature-Based Models

As the limitations of purely rule-based systems became evident, the focus shifted towards machine learning approaches that could learn to recognize entities from annotated data.

Hidden Markov Models

Early probabilistic models using states to represent entity classes and learning transition probabilities.

Decision Trees

Used to derive features and make classification decisions based on hierarchical rules.

Conditional Random Fields

Discriminative probabilistic framework for sequence labeling that overcame HMM limitations.

Maximum Entropy

Classifiers that model probability distributions based on constraints from training data.

Mathematical Foundations

Behind effective NER systems lie sophisticated mathematical frameworks that enable machines to identify and classify named entities in text.

Probability Theory

Statistical NER models rely on probability distributions to capture the likelihood of words belonging to specific entity classes.

Sequence Labeling

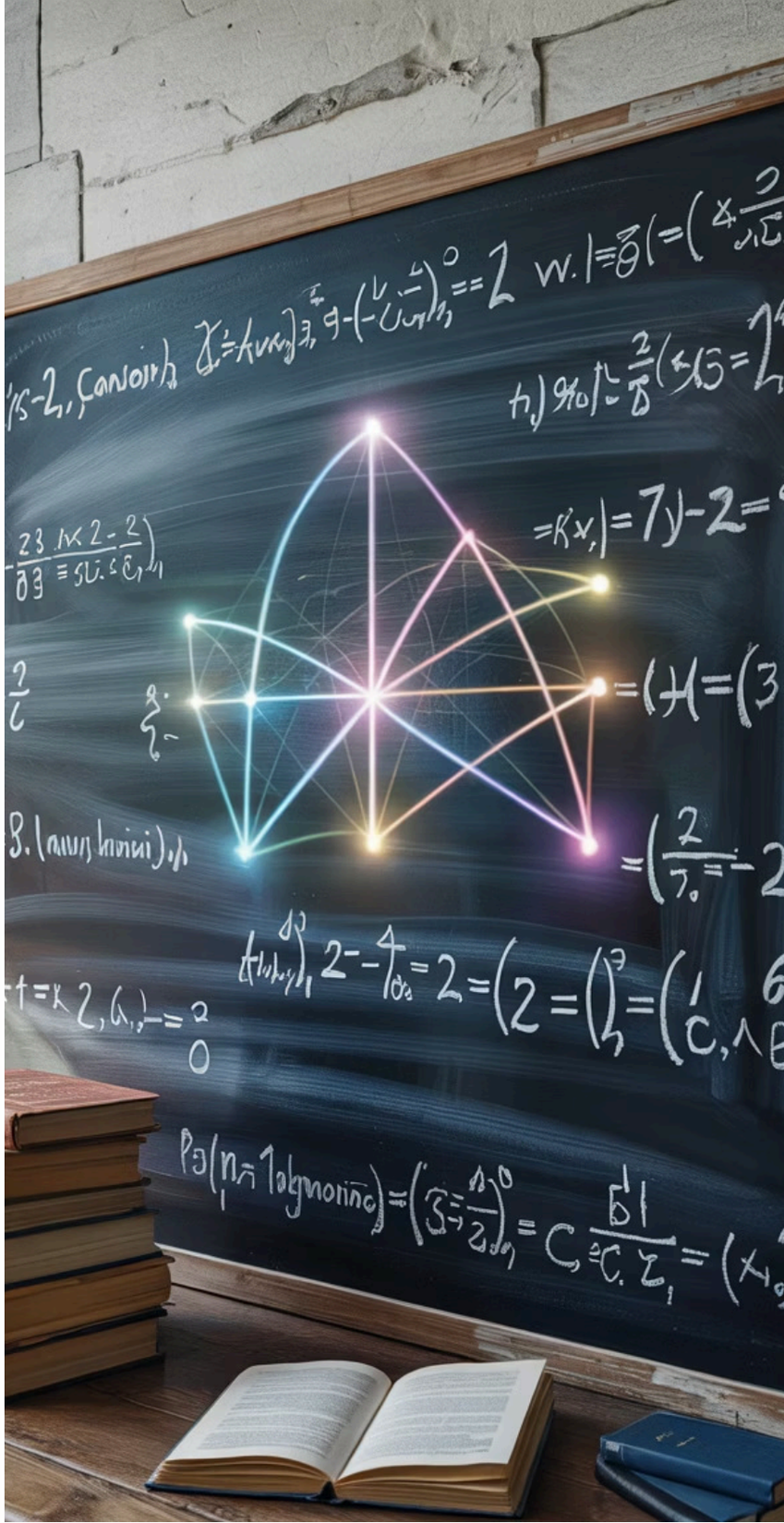
NER mathematically represents text as sequences where each token must be assigned the optimal entity tag.

Optimization Algorithms

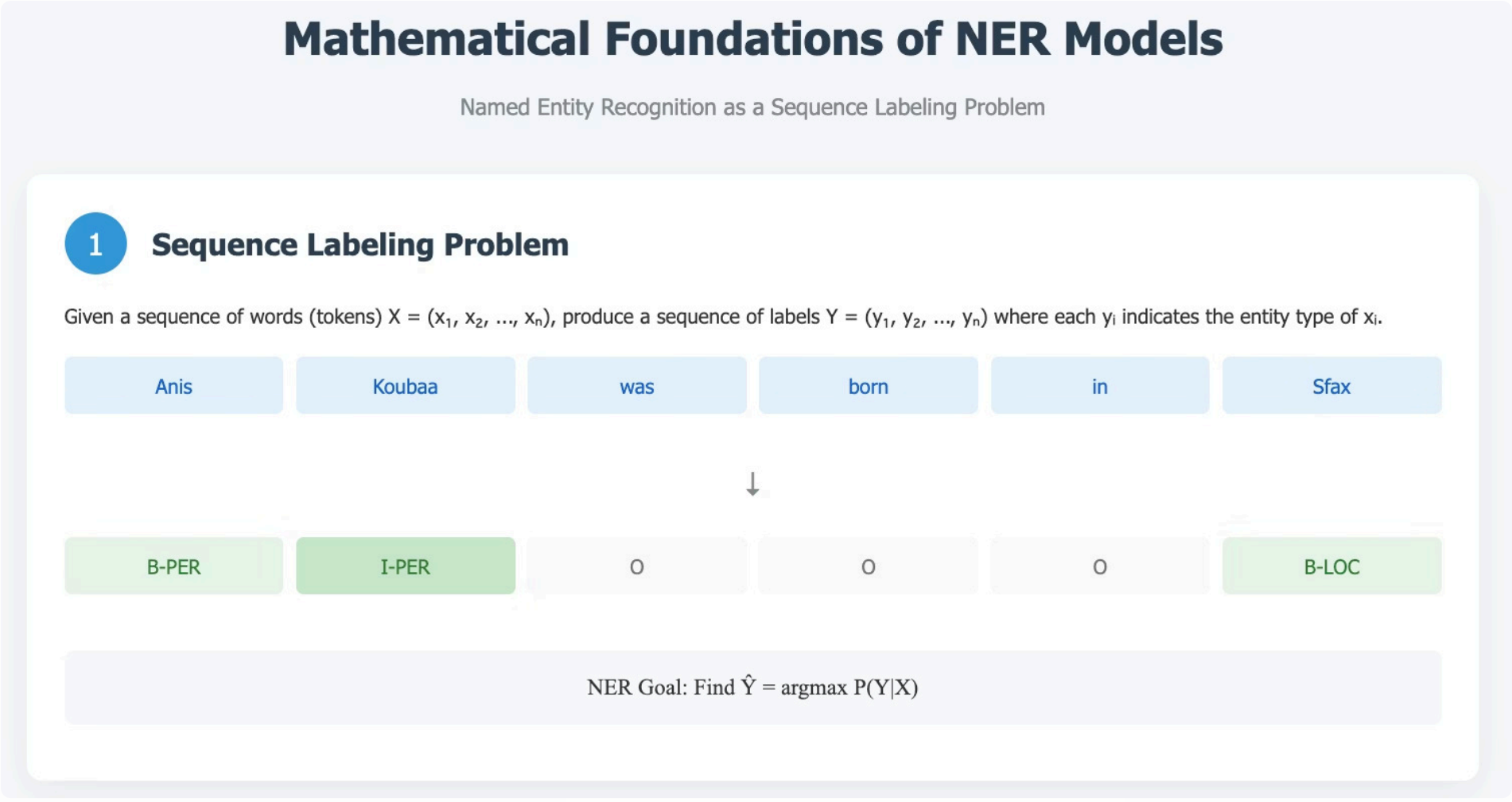
Models use gradient descent and other techniques to minimize error and maximize classification accuracy.

Vector Representations

Modern NER systems transform words into dense mathematical vectors that capture semantic meaning.



Named Entity Recognition as a Sequence Labeling Problem



IOB Tagging Format

2 IOB Tagging Format

B-

Beginning of a named entity

I-

Inside a named entity

O

Outside any named entity

Common entity types include:

- **PER**: Person (e.g., B-PER, I-PER)
- **ORG**: Organization (e.g., B-ORG, I-ORG)
- **LOC**: Location (e.g., B-LOC, I-LOC)
- **DATE**: Temporal expressions (e.g., B-DATE, I-DATE)

Hidden Markov Models

3 Hidden Markov Models (HMMs)

HMMs model the joint probability P(X,Y) of observations X and hidden states Y:

$$P(X,Y) = P(y_1) \cdot \prod^{k}_{i=2} P(y_i|y_{i-1}) \cdot \prod^{k}_{i=1} P(x_i|y_i)$$

Initial State Probability → Transition Probabilities → Emission Probabilities

Key components:

- **Initial state probability**: $P(y_1)$
- **Transition probability**: $P(y_i|y_{i-1})$ - probability of transitioning from one tag to another
- **Emission probability**: $P(x_i|y_i)$ - probability of observing word x_i given tag y_i

Conditional Random Fields (CRFs)

4 Conditional Random Fields (CRFs)

CRFs directly model the conditional probability P(Y|X) using feature functions:

$$P(Y|X) = 1/Z(X) \cdot \exp(\sum_i \sum_j \lambda_j f_j(y_{i-1}, y_i, X, i))$$

Where:

- **Z(X)**: Normalization factor
- $\boldsymbol{\lambda}_j$: Weight for feature f_j
- $\mathbf{f}_j$: Feature function capturing aspects of transition $y_{i-1} \rightarrow y_i$ and features of X around position i

CRFs can incorporate rich, overlapping features without independence assumptions:

- Word identity, capitalization, prefixes/suffixes
- Dictionary lookups, POS tags
- Context window features (surrounding words)

Model Comparison & Decoding

5 Model Comparison & Decoding

HMM Advantages

- Simple probabilistic model
- Fast training and decoding
- Well-established mathematical foundation

CRF Advantages

- Overcomes label bias problem
- Can use rich, overlapping features
- Directly optimizes P(Y|X)
- State-of-the-art before deep learning

Decoding: Both models typically use the Viterbi algorithm to find the most likely sequence $Y^* = \text{argmax } P(Y|X)$.

$$Y^* = \text{Viterbi}(X, \text{model parameters})$$

Mathematical Foundations: **Hidden Markov Models**

Hidden Markov Models (HMMs) are **generative probabilistic models** that assume a **sequence of hidden states** (entity labels) that **emit observable outputs (words)** with certain **probabilities**.

Key Components

- States for each entity tag (PER, LOC, ORG, O)
- Transition probabilities between states
- Emission probabilities of words given states

Joint Probability

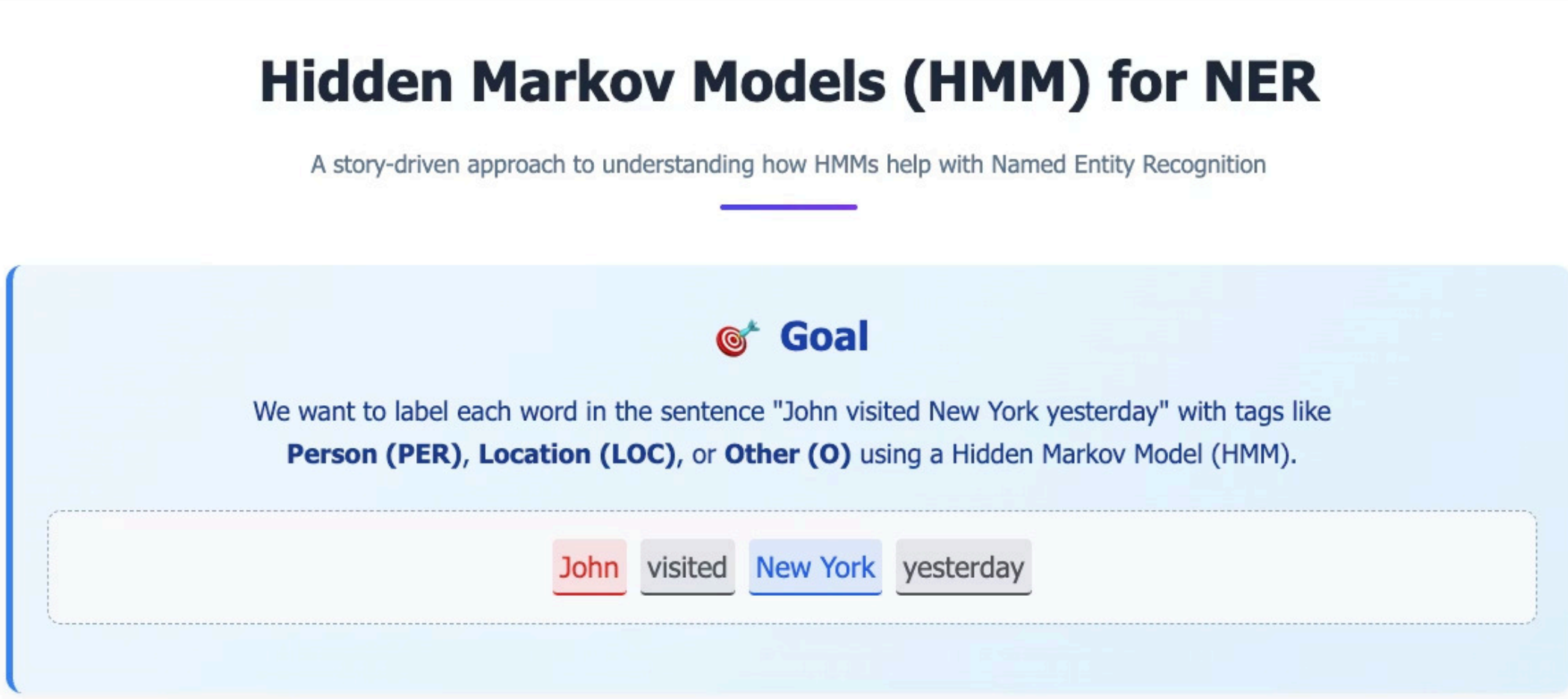
HMMs assign a joint probability to a word sequence X and label sequence Y as:

$$P(X,Y) = \prod_{i=1}^{n} P(x_i|y_i) \cdot P(y_i|y_{i-1})$$

Limitations

- Generative nature requires simplifying assumptions
- Difficulty modeling overlapping features
- Limited to local context (Markov assumption)

Hidden Markov Models (HMM) for NER



What is an HMM?

1 What is an HMM?

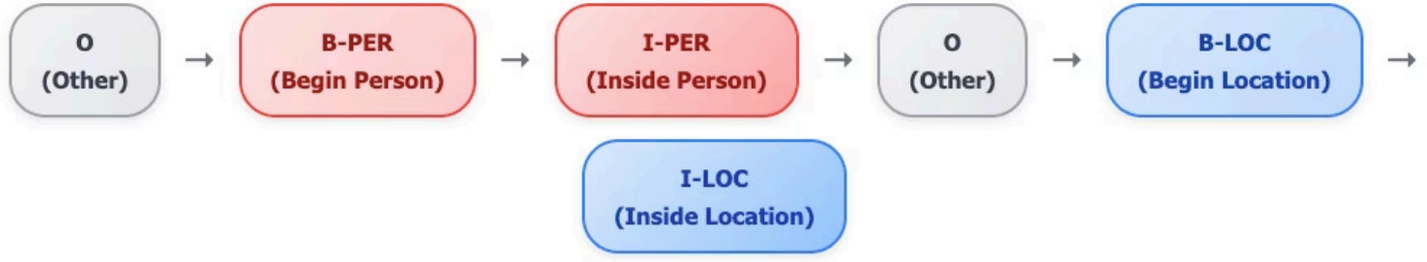
An HMM (Hidden Markov Model) is a statistical model that helps us label words in text.

🤖 The Robot Analogy

Think of an HMM as a **robot playing a guessing game**. It can't directly see whether a word is a person or location — it can only guess based on:

- **Previous guesses** (transition probabilities) - What label usually comes after the current one?
- **How likely the word fits a label** (emission probabilities) - How often does this word appear as a person name or location?

Possible States (Labels) in Our Model



The robot jumps between these states as it reads through the text

Input

2 Input: Our Example Sentence

We'll walk through this sentence **word by word**, trying to guess the best label for each word.

"John visited New York yesterday"

Our goal is to correctly identify "John" as a person and "New York" as a location.

Probabilities as Hints

3 Probabilities as Hints

The HMM uses two types of probabilities to make its guesses:

🎲 Emission Probabilities

These tell us **how likely a word is** if we are in a certain state.

Example:

- "John" is very likely to be B-PER (0.1), but unlikely to be O (0.001).
- "visited" is likely to be O (0.05), but unlikely to be B-PER (0.001).

🔄 Transition Probabilities

These tell us **how likely it is to go from one state to another**.

Example:

- After B-PER, we are likely to go to I-PER (0.8).
- After B-LOC, we are likely to go to I-LOC (0.8).

💡 Everyday Analogy

"If I say 'John', it's probably the beginning of a person name. If I then say 'Smith', it's probably still the same person. But if I say 'John visited', then 'visited' is probably not part of the person's name."

🎲 Emission Probabilities

These tell us how likely a word is if we are in a certain state.

Word	B-PER	B-LOC	O
John	**0.1**	0.002	0.001
visited	0.001	0.001	**0.1**
New	0.01	**0.08**	0.005

Note: The table shows simplified probability values.

🔄 Transition Probabilities

These tell us how likely it is to go from one state to another.

From / To	B-PER	I-PER	O
B-PER	0.01	**0.8**	0.1
O	**0.2**	0.01	0.5
B-LOC	0.01	0.01	0.2

Note: The table shows simplified probability values.

Real-world analogy:

"If I say 'John', it's probably the beginning of a person name. If I then say 'Smith', it's probably still the same person."

"If I say 'visited', it's probably not part of any entity - it's just a regular verb (O tag)."

The Viterbi Algorithm Without the Math

4 Viterbi Algorithm Without the Math

The Viterbi algorithm is like **trying all paths** through the states and **keeping the best scoring one**.

For "John"

Try all states: B-PER fits best because "John" is commonly a person name.

For "visited"

From B-PER, try going to each possible state. O fits best because "visited" is a verb, not part of the person's name.

For "New"

From O, try all states. B-LOC fits best because "New" often starts location names.

For "York"

From B-LOC, try all states. I-LOC fits best because "York" is continuing the location name.

For "yesterday"

From I-LOC, try all states. O fits best because "yesterday" is a time expression, not part of the location.

The Best Path Through the Sequence

Word	John	visited	New	York	yesterday
Best State	B-PER	O	B-LOC	I-LOC	O

The table shows the most likely sequence of labels (states) for our sentence.

The Viterbi algorithm is like **trying all paths** through the states and **keeping the best scoring one**.

Instead of complicated formulas, think of it this way:

1. Start with "John" → Try all states: which one fits best?
2. Then go to "visited" → From each previous state, try jumping to new states and score the path.
3. Keep doing this and **remember which path gave the best score**.

State / Word	John	visited	New	York	yesterday
B-PER	**0.1**	0.0008	0.00016	0.0000032	0.00000064
I-PER	0.02	0.0016	0.00008	0.000016	0.0000032
O	0.05	**0.01**	0.001	0.0001	**0.00005**
B-LOC	0.01	0.001	**0.008**	0.00016	0.000016
I-LOC	0.005	0.0004	0.00002	**0.0064**	0.000128

Think of it like GPS navigation:

If you want to go from New York to Los Angeles, there are many possible routes. The Viterbi algorithm is like exploring all routes simultaneously, but at each city (word), keeping only the best route to that city so far. At the end, you trace back through the best final path.

Simplified mental process of the algorithm:

"John"	"visited"	"New"	"York"	"yesterday"
Looks like a person!	Just a verb	Start of a place?	Continuing a place	Just a time word
B-PER: 0.1 ✓	O: 0.01 ✓	B-LOC: 0.008 ✓	I-LOC: 0.0064 ✓	O: 0.00005 ✓

Traceback and Final Tags

How Are Emission and Transition Probabilities Calculated for HMMs?

How Are Emission and Transition Probabilities Calculated for HMMs?

1 You Need a Tagged Dataset (Supervised Training)

To calculate emission and transition probabilities for a Hidden Markov Model (HMM), we need a **labeled training corpus** where each word has a known entity label (like O, B-PER, I-LOC, etc.).

Sentence	Tags
John visited New York	B-PER O B-LOC I-LOC
Alice lives in Paris	B-PER O O B-LOC
Yesterday, Peter left London	O B-PER O O B-LOC

Why do we need labeled data?
The HMM needs to learn patterns like:

- "John" is likely to be a person (B-PER)
- After a B-LOC tag, an I-LOC tag often follows (like in "New York")
- Words like "visited" and "lives" are typically not part of named entities (O tag)

These patterns are captured in the transition and emission probabilities.

2 Transition Probabilities

Transition probabilities tell us how likely it is to move from one tag to another (e.g., from B-PER to I-PER).

Formula:

$$P(t_j|t_i) = \frac{\text{Count}(t_i \rightarrow t_j)}{\text{Count}(t_i)}$$

Where:

- **Count($t_i \rightarrow t_j$)**: Number of times tag t_j follows tag t_i
- **Count(t_i)**: Total number of times tag t_i appears

Counting Transitions: B-PER → I-PER and B-PER → O

John	Smith	visited	New	York
B-PER	I-PER	O	B-LOC	I-LOC

Here we count one B-PER → I-PER transition

Alice	lives	in	Paris
B-PER	O	O	B-LOC

Here we count one B-PER → O transition

Total B-PER tags	B-PER → I-PER transitions	B-PER → O transitions
100	80	20
Count(B-PER) = 100	Count(B-PER → I-PER) = 80	Count(B-PER → O) = 20

How Are Emission and Transition Probabilities Calculated for HMMs?

1 You Need a Tagged Dataset (Supervised Training)

To calculate emission and transition probabilities for a Hidden Markov Model (HMM), we need a **labeled training corpus** where each word has a known entity label (like O, B-PER, I-LOC, etc.).

Sentence	Tags
John visited New York	B-PER O B-LOC I-LOC
Alice lives in Paris	B-PER O O B-LOC
Yesterday, Peter left London	O B-PER O O B-LOC

Why do we need labeled data?
The HMM needs to learn patterns like:

- "John" is likely to be a person (B-PER)
- After a B-LOC tag, an I-LOC tag often follows (like in "New York")
- Words like "visited" and "lives" are typically not part of named entities (O tag)

These patterns are captured in the transition and emission probabilities.

Total B-PER tags	B-PER → I-PER transitions	B-PER → O transitions
100	80	20
Count(B-PER) = 100	Count(B-PER → I-PER) = 80	Count(B-PER → O) = 20

Calculating Transition Probabilities

P(I-PER | B-PER) = Count(B-PER → I-PER) / Count(B-PER)

= 80 / 100 = **0.8**

P(O | B-PER) = Count(B-PER → O) / Count(B-PER)

= 20 / 100 = **0.2**

This tells us that after seeing a B-PER (beginning of person) tag, there's an 80% chance the next tag will be I-PER (inside person) and a 20% chance it will be O (outside any entity).

Important: You compute transition probabilities for **every pair of tags** in your tag set (B-PER, I-PER, B-LOC, I-LOC, O, etc.).

How Are Emission and Transition Probabilities Calculated for HMMs?

1 You Need a Tagged Dataset (Supervised Training)

To calculate emission and transition probabilities for a Hidden Markov Model (HMM), we need a **labeled training corpus** where each word has a known entity label (like O, B-PER, I-LOC, etc.).

Sentence	Tags
John visited New York	B-PER O B-LOC I-LOC
Alice lives in Paris	B-PER O O B-LOC
Yesterday, Peter left London	O B-PER O O B-LOC

Why do we need labeled data?
The HMM needs to learn patterns like:

- "John" is likely to be a person (B-PER)
- After a B-LOC tag, an I-LOC tag often follows (like in "New York")
- Words like "visited" and "lives" are typically not part of named entities (O tag)

These patterns are captured in the transition and emission probabilities.

3 Emission Probabilities

Emission probabilities tell us how likely a specific word is to be generated from a particular tag (e.g., how likely is "John" if the tag is B-PER).

Formula:

$$P(w_k|t_j) = \frac{\text{Count}(t_j \text{ emits } w_k)}{\text{Count}(t_j)}$$

Where:

- **Count(t_j emits w_k)**: Number of times word w_k appears with tag t_j
- **Count(t_j)**: Total number of times tag t_j appears

Counting Emissions: B-PER emits "John"

John	visited	New	York
B-PER	O	B-LOC	I-LOC

Total B-PER tags

1000

Count(B-PER) = 1000

"John" with B-PER tag

100

Count(B-PER emits "John") = 100

How Are Emission and Transition Probabilities Calculated for HMMs?

1 You Need a Tagged Dataset (Supervised Training)

To calculate emission and transition probabilities for a Hidden Markov Model (HMM), we need a **labeled training corpus** where each word has a known entity label (like O, B-PER, I-LOC, etc.).

Sentence	Tags
John visited New York	B-PER O B-LOC I-LOC
Alice lives in Paris	B-PER O O B-LOC
Yesterday, Peter left London	O B-PER O O B-LOC

Why do we need labeled data?
The HMM needs to learn patterns like:

- "John" is likely to be a person (B-PER)
- After a B-LOC tag, an I-LOC tag often follows (like in "New York")
- Words like "visited" and "lives" are typically not part of named entities (O tag)

These patterns are captured in the transition and emission probabilities.

Total B-PER tags

1000

Count(B-PER) = 1000

"John" with B-PER tag

100

Count(B-PER emits "John") = 100

Calculating Emission Probability

P("John" | B-PER) = Count(B-PER emits "John") / Count(B-PER)

= 100 / 1000 = **0.1**

This tells us that when we see a B-PER tag, there's a 10% chance the corresponding word is "John". We would do similar calculations for all other words that appear with the B-PER tag.

Important: You compute emission probabilities for **every word-tag pair** in your training data.

How Are Emission and Transition Probabilities Calculated for HMMs?

1 You Need a Tagged Dataset (Supervised Training)

To calculate emission and transition probabilities for a Hidden Markov Model (HMM), we need a **labeled training corpus** where each word has a known entity label (like O, B-PER, I-LOC, etc.).

Sentence	Tags
John visited New York	B-PER O B-LOC I-LOC
Alice lives in Paris	B-PER O O B-LOC
Yesterday, Peter left London	O B-PER O O B-LOC

Why do we need labeled data?
The HMM needs to learn patterns like:

- "John" is likely to be a person (B-PER)
- After a B-LOC tag, an I-LOC tag often follows (like in "New York")
- Words like "visited" and "lives" are typically not part of named entities (O tag)

These patterns are captured in the transition and emission probabilities.

4 Smoothing (Handling Unseen Data)

One challenge with calculating probabilities directly from counts is handling words or transitions that haven't been seen in the training data. For these cases, we use **smoothing techniques**.

The Zero Probability Problem
If a word like "Anthropic" never appeared with the B-ORG tag in your training data, the emission probability P("Anthropic" | B-ORG) would be 0. This is problematic because:

- Even one zero probability in a sequence makes the entire sequence probability zero
- New words should still have some chance of being entities

Laplace (Add-1) Smoothing:

$$P(w_k|t_j) = \frac{\text{Count}(t_j \text{ emits } w_k) + 1}{\text{Count}(t_j) + |V|}$$

Where |V| is the size of your vocabulary (total number of unique words).

Smoothing Example
Suppose "Anthropic" never appears with B-ORG in training:

Count(B-ORG emits "Anthropic") = 0

Count(B-ORG) = 500

Vocabulary size |V| = 10,000

Without smoothing:

P("Anthropic" | B-ORG) = 0 / 500 = 0

With Laplace smoothing:

P("Anthropic" | B-ORG) = (0 + 1) / (500 + 10,000) = 1 / 10,500 ≈ 0.000095

Now "Anthropic" has a small but non-zero probability of being generated from the B-ORG tag, which prevents the whole sequence probability from becoming zero.

How Are Emission and Transition Probabilities Calculated for HMMs?

5 Summary

Type	Formula	Intuition
Transition Probability	$P(t_j \mid t_i) = \frac{\text{Count}(t_i \rightarrow t_j)}{\text{Count}(t_i)}$	"How often does tag j follow tag i?"
Emission Probability	$P(w_k \mid t_j) = \frac{\text{Count}(t_j \text{ emits } w_k)}{\text{Count}(t_j)}$	"How often does tag j generate the word w?"

Key Takeaways:

- Both emission and transition probabilities are calculated from **frequencies in labeled training data**
- Transition probabilities model the **tag sequence patterns** (e.g., person names often continue across multiple words)
- Emission probabilities model **which words belong to which entity types** (e.g., "John" is likely a person)
- Smoothing handles **unseen words and transitions** by giving them small non-zero probabilities
- These probabilities are the key parameters of your HMM and will be used by the Viterbi algorithm for decoding (finding the most likely tag sequence)

Pro Tip: Modern NER systems often use more complex models like BiLSTM-CRF or transformer-based models, but understanding HMMs provides a solid foundation for how sequence labeling works.

Conditional Random Fields: A Breakthrough

Conditional Random Fields (CRFs) directly model the conditional probability P(Y|X) of a label sequence given the observation sequence, overcoming many limitations of HMMs.

Discriminative Approach

Models P(Y|X) directly instead of joint probability P(X,Y)

Feature Functions

Uses feature functions that can inspect current/previous labels and entire input

Weighted Features

Learns weights for features from training data

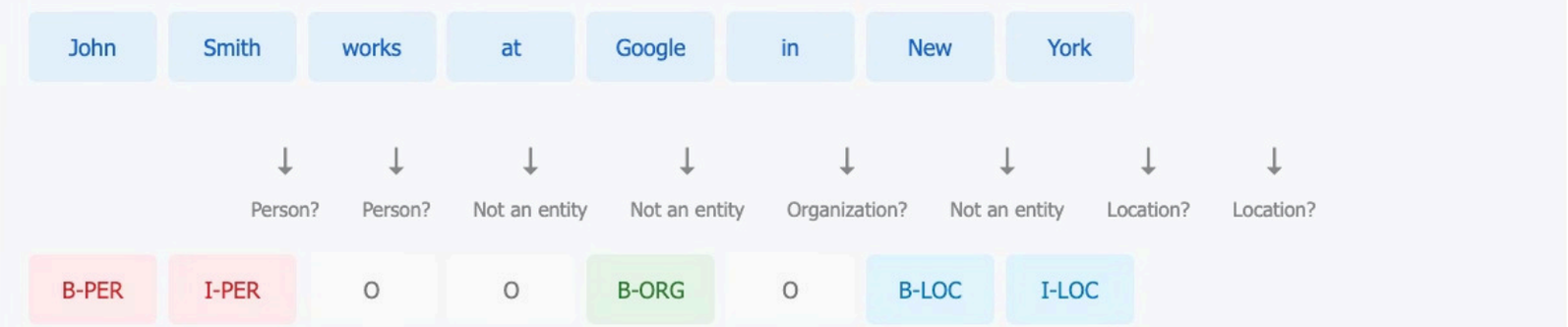
Global Normalization

Avoids label bias by normalizing over entire sequences

Conditional Random Fields

1 Set the Stage with a Problem

"Imagine you're trying to tag names of people and places in a sentence. But to guess the correct label for a word, just looking at the word itself isn't enough—you also need to know its neighbors, its shape, and even what the previous word was tagged as."



Can we confidently label 'John' as a person without looking at 'Smith'? Or 'York' without seeing 'New'?

This shows why **sequence labeling** is hard — we need **context**, **patterns**, and **rules**.

The Context Challenge:

Words like "Smith" could be a person's name or a profession (blacksmith). "New" alone isn't a location, but "New York" is. "Washington" could be a person or a place depending on context. We need to look at the entire sentence to make good decisions.

HMM vs CRF — Two Types of Detectives

2 HMM vs CRF — Two Types of Detectives

🤔 HMM: The Old-School Detective

He tries to **recreate how the sentence was generated** by guessing the hidden path (tags) that *produced* the words. He's limited — he can only look at the current word and guess from past.

- Assumes independence between observations
- Can't use rich features like word shape or dictionaries
- Works "one step at a time" with only limited hindsight
- Asks: "What hidden states would generate these words?"

👩‍⚖️ CRF: The Modern Judge

She looks at the **whole sentence**, all the **evidence**, and directly **decides the best label sequence**, using logic and many clues (features). She doesn't care about how the sentence was made—only what it *means*.

- Uses all available information at once
- Can incorporate hundreds of overlapping features
- Considers the global picture before making decisions
- Asks: "Given these words, what's the most likely label sequence?"

A Real-World Analogy:

HMM is like a crime investigator who tries to reconstruct what happened at the crime scene step by step. "First the perpetrator entered through the door, then took the vase..." He's focused on the sequence of events that led to the evidence we see.

CRF is like a modern forensic team with DNA tests, fingerprint analysis, and many other tools. They don't need to recreate the crime—they look at all evidence together and make their judgment directly: "The evidence points to this person."

Feature Functions

3 Feature Functions

What clues help us know a word is a person's name?

Clues that help identify named entities:

Word	Clue: Capitalized	Clue: In names list	Clue: Previous tag	Clue: Next word
John	✅	✅	—	Smith
Google	✅	❌ (not a person)	O (at)	in
New	✅	❌	O (in)	York
York	✅	❌	B-LOC (New)	—

Each clue is a **feature function**. Some clues help us a lot (like names list), others less so (like word length). CRFs use **hundreds of such clues** and learn which ones are more useful by assigning them **weights**.

Why CRFs are powerful:

CRFs can use many types of features that HMMs can't:

- **Word shape features:** Is it ALL CAPS? Capitalized? Contains numbers?
- **Dictionary features:** Is it in a list of company names? Person names?
- **Context window features:** What are the previous and next 2-3 words?
- **Character n-gram features:** Does it end with "son", "ville", or "burg"?
- **Document-level features:** Has this word appeared earlier in the document?

Weighted Evidence: Some Clues Matter More

4 Weighted Evidence: Some Clues Matter More

The Detective Analogy:

"Let's say we have multiple detectives. Each detective finds a clue and gives it a confidence score. One says 'This word is capitalized' — good clue! Another says 'It ends with "ing"' — not helpful. CRF **adds up all these clues, multiplies them by how important they are**, and chooses the best label sequence."

Example: Weighting features for "John"

Feature	Active?	Weight (λ)	Score	Weight Strength
Word is capitalized	Yes	+2.3	+2.3	
Word is in names list	Yes	+3.7	+3.7	
Word ends in "ing"	No	-2.0	0	
Previous tag is "at"	No	+1.0	0	
		Total score:	**+6.0**	**Strong case for B-PER**

How weights are like detective credibility:

Some detectives (features) have better track records than others. If Detective Williams always correctly identifies persons based on name dictionaries (weight +3.7), we trust her more than Detective Johnson who often makes mistakes by just looking at capitalization (weight +2.3).

CRF's training phase is like evaluating each detective's past performance to decide how much to trust their opinion.

CRF Mathematical Formulation

$$P(Y|X) = \mathbf{1/Z(X)} \cdot \exp(\sum_i \sum_j \boldsymbol{\lambda_j f_j}(y_{i-1}, y_i, X, i))$$

f_j: Feature Functions

The detectives that identify clues (like "word is capitalized" or "word in dictionary")

λ_j: Weights

How much we trust each detective's opinion

Z(X): Normalizer

Makes probabilities add up to 1 across all possible tag sequences

The detective's final report:

Think of the CRF as a team of detectives who submit a final report about how to label the sentence. The **feature functions** are the individual detectives' findings, the **weights** show how credible each detective is, and the **normalizer** ensures the final probabilities make mathematical sense (adding up to 100%).

The formula simply says: "We combine all the weighted evidence and pick the most likely explanation."

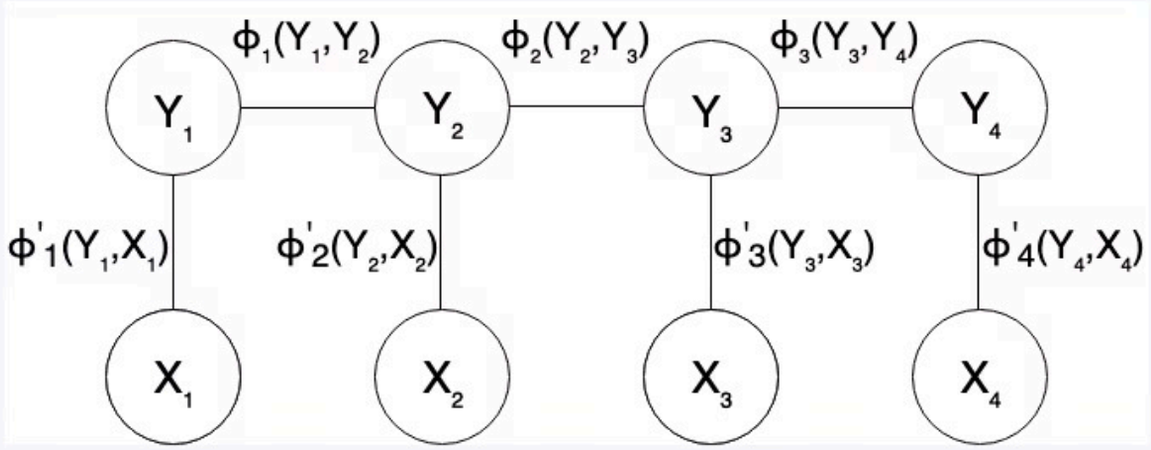
CRF Mathematical Formulation

The Math, Simplified

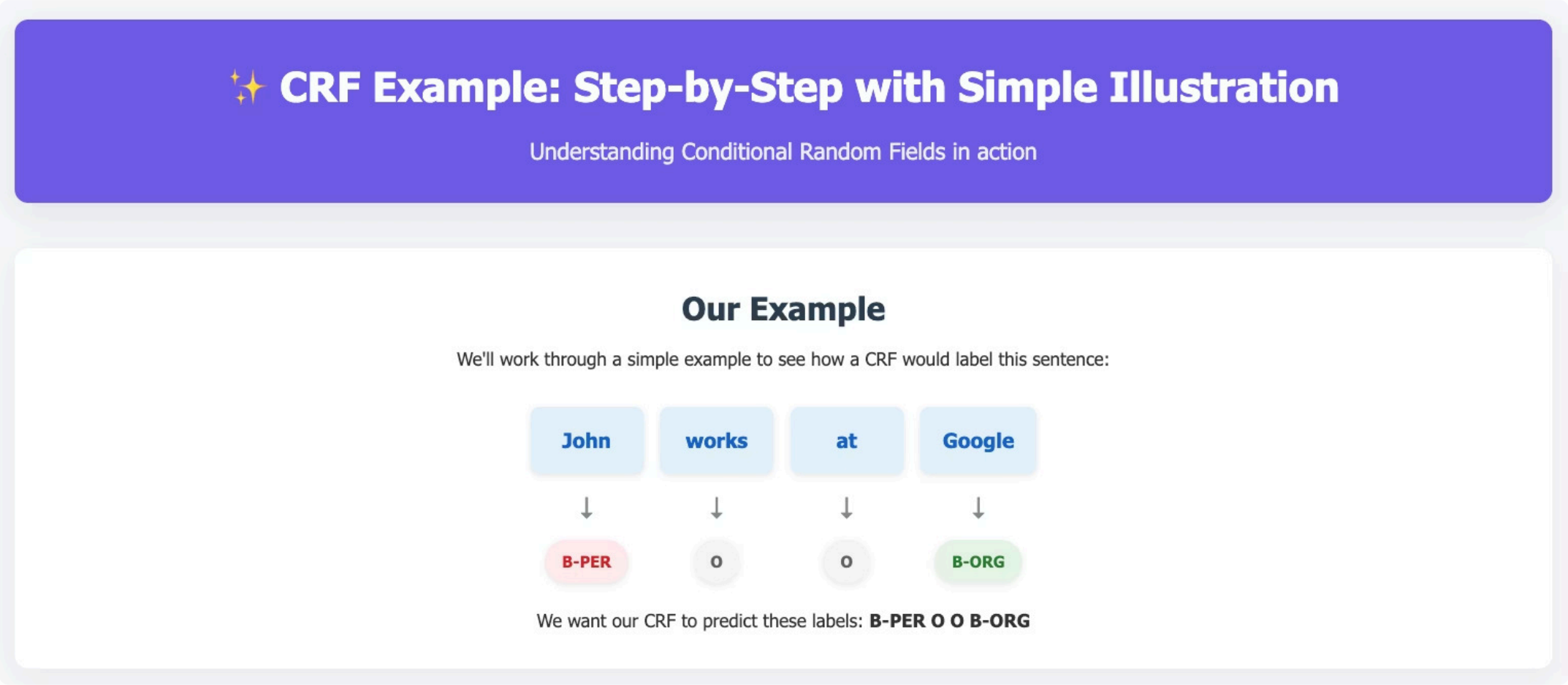
CRF loops over the sentence. At each word, it activates all the relevant clues (features), multiplies each by how important it is (its weight), and adds them up. Then it chooses the best label sequence across the whole sentence.

$$P(Y|X) = \frac{1}{Z(X)} \exp\left(\sum_i \sum_j \lambda_j f_j(y_{i-1}, y_i, X, i)\right)$$

f_j : Feature functions (detectives) λ_j : Weights (how much we trust each detective) $Z(X)$: Normalizer to make probabilities add up to 1



sCRF Example: Step-by-Step Illustration



CRF Example: Step-by-Step Illustration

1 Define Feature Functions (Clues)

At each position *i*, we use **features** (our detective clues). Let's define just a few simple ones:

Feature Function	Description	Example
f_1	Word is capitalized	e.g., "John" is capitalized
f_2	Current word is in names dictionary	"John" → Yes
f_3	Current tag is B-PER and next tag is I-PER	Encourages name continuation
f_4	Current word is "at"	Helps suggest O tag

Note: Real CRFs would use dozens or hundreds of features, but we're keeping it simple for this example.

CRF Example: Step-by-Step Illustration

2 Assign Weights to Features

Each feature is assigned a weight λ (lambda) that represents how important that feature is for making predictions. These weights are learned during training, but we'll use the following values for our example:

Feature	Weight (λ)	Interpretation
f_1: word is capitalized	+2.0	Capitalization is a good indicator of named entities
f_2: word in names dictionary	+3.0	Being in a names list is a very strong indicator for person entities
f_3: B-PER followed by I-PER	+1.0	Encourages multi-word person names
f_4: word is "at"	-1.5	The word "at" is unlikely to be part of an entity

Positive weights increase the likelihood of a particular label when the feature is active. **Negative weights** decrease the likelihood of a label when the feature is active.

CRF Example: Step-by-Step Illustration

3 Activate Features for a Label Sequence

Suppose we try the label sequence: **B-PER, O, O, B-ORG**
We now calculate the total score using:

$$\text{Score} = \sum_i \sum_j \lambda_j f_j(y_{i-1}, y_i, x, i)$$

Let's go word-by-word to calculate this score:

Word: "John" **Label: B-PER**

- ✓ Capitalized → $f_1 = 1$
- ✓ In names dictionary → $f_2 = 1$

Partial score: $2.0 \times 1 + 3.0 \times 1 = \mathbf{5.0}$

Word: "works" **Label: O**

- ✗ Not capitalized → $f_1 = 0$
- ✗ Not in name list → $f_2 = 0$

Partial score: 0

Word: "works" **Label: O**

- ✗ Not capitalized → $f_1 = 0$
- ✗ Not in name list → $f_2 = 0$

Partial score: 0

Word: "at" **Label: O**

- ✗ Not capitalized → $f_1 = 0$
- ✗ Not in name list → $f_2 = 0$
- ✓ Word is "at" → $f_4 = 1$

Partial score: $-1.5 \times 1 = \mathbf{-1.5}$

Word: "Google" **Label: B-ORG**

- ✓ Capitalized → $f_1 = 1$
- ✗ Not in name dictionary → $f_2 = 0$

Partial score: $2.0 \times 1 = \mathbf{2.0}$

CRF Example: Step-by-Step Illustration

4 Add All Scores

$$\text{Total Unnormalized Score} = 5.0 + 0 + (-1.5) + 2.0 = \mathbf{5.5}$$

$$P(Y|X) = \frac{e^{5.5}}{Z(X)}$$

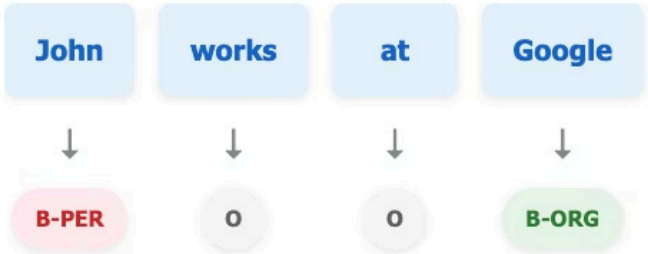
Where:

- $\mathbf{e^{5.5}} \approx 244.7$ is the unnormalized score for this sequence
- **Z(X)** is the sum of e^{score} over **all possible label sequences**

The normalizer Z(X) ensures that all sequence probabilities sum to 1. It's calculated by summing the unnormalized scores (e^{score}) for every possible label sequence for our sentence.
For our 4-word sentence with 4 possible labels (B-PER, I-PER, B-ORG, O), there are $4^4 = 256$ possible label sequences! In practice, efficient algorithms like the forward-backward algorithm are used to compute this.

5 Select the Best Sequence

The CRF evaluates **all possible tag sequences** for the sentence, calculates their score using the activated features, and picks the one with the **highest probability** after normalization.
In our example, the sequence B-PER, O, O, B-ORG received a score of 5.5, which gives an unnormalized probability of $e^{5.5} \approx 244.7$. After normalizing, this becomes $P(Y|X) = e^{5.5}/Z(X)$.
If this is the highest probability among all possible label sequences, the CRF would predict:



CRF NER Summary

✅ Visual Summary Box

Sentence: John works at Google

Prediction: B-PER O O B-ORG

Features (✓ = active)	John	works	at	Google
Capitalized	✓	✗	✗	✓
In name list	✓	✗	✗	✗
Word = "at"	✗	✗	✓	✗
Feature Score:	**+5.0**	**0**	**-1.5**	**+2.0**

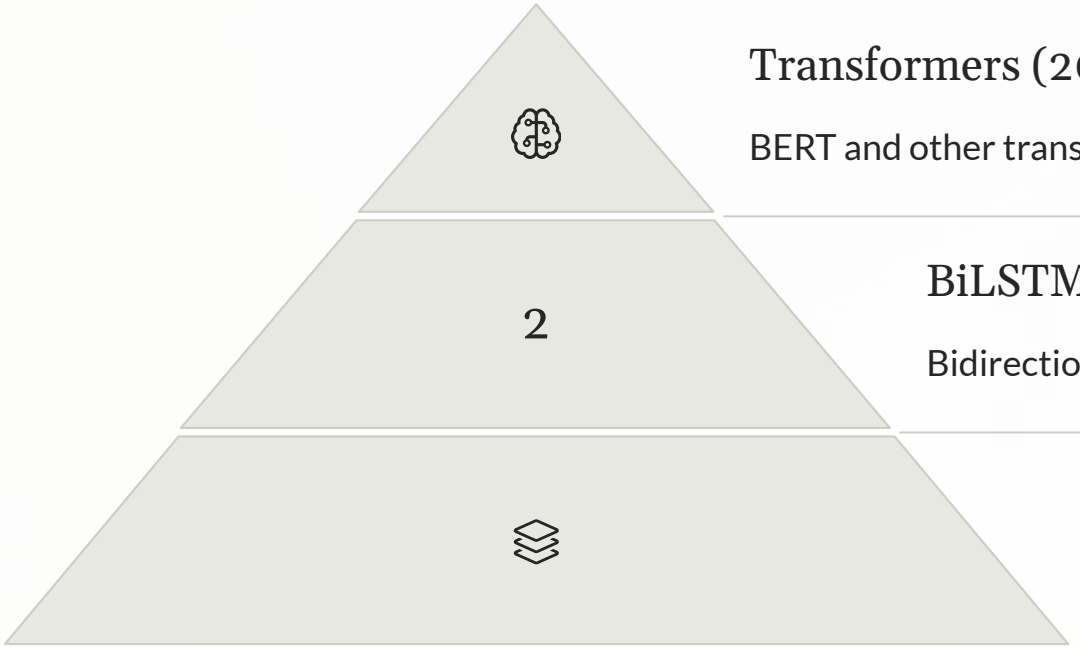
$$\text{Total Score} = 5.5 \rightarrow \exp(5.5) \approx 244.7$$

$$\text{Final Probability} = \exp(5.5) / Z(X)$$

The CRF has successfully identified "John" as a person (B-PER) and "Google" as an organization (B-ORG) by considering the features and their weights. The sequence B-PER O O B-ORG has the highest probability among all possible tag sequences for this sentence.

Neural Network Revolution in NER

The true transformation of NER occurred with the rise of neural networks and deep learning, which significantly reduced the need for manual feature engineering.



Transformers (2019+)

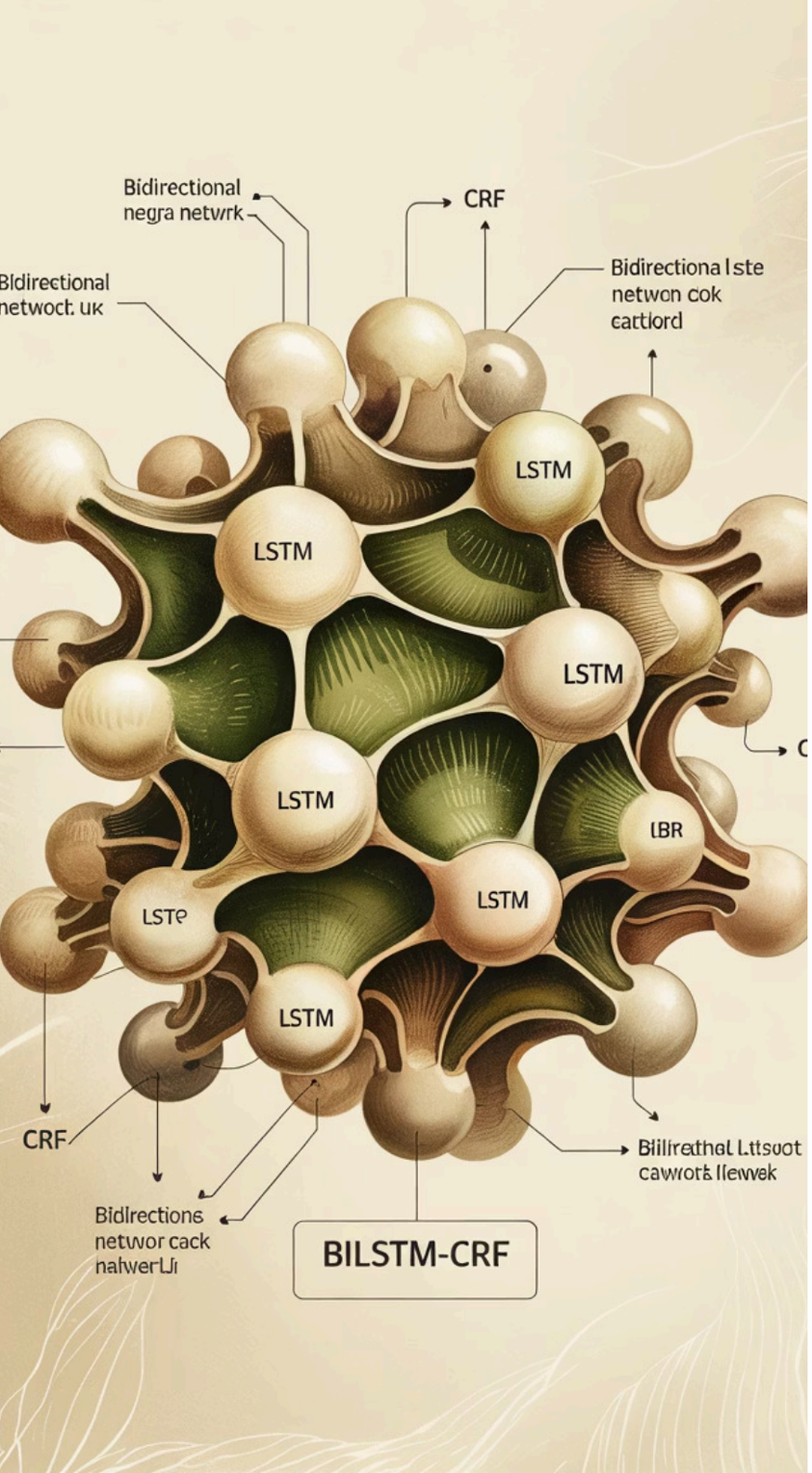
BERT and other transformer models with self-attention mechanisms

BiLSTM-CRF (2015-2018)

Bidirectional LSTMs with CRF output layers

CNN & Word Embeddings (2011-2014)

Convolutional networks with learned word representations



BiLSTM-CRF Architecture

The Bidirectional LSTM + CRF model has been a staple for NER since the mid-2010s, combining the contextual power of bidirectional LSTMs with the structured prediction capabilities of CRFs.

A

Character-Level Encoder

Captures morphological and orthographic features using CNN or LSTM over characters of each word.

⇔

Word-Level BiLSTM

Processes the sequence forward and backward to produce context-aware representations for each word.

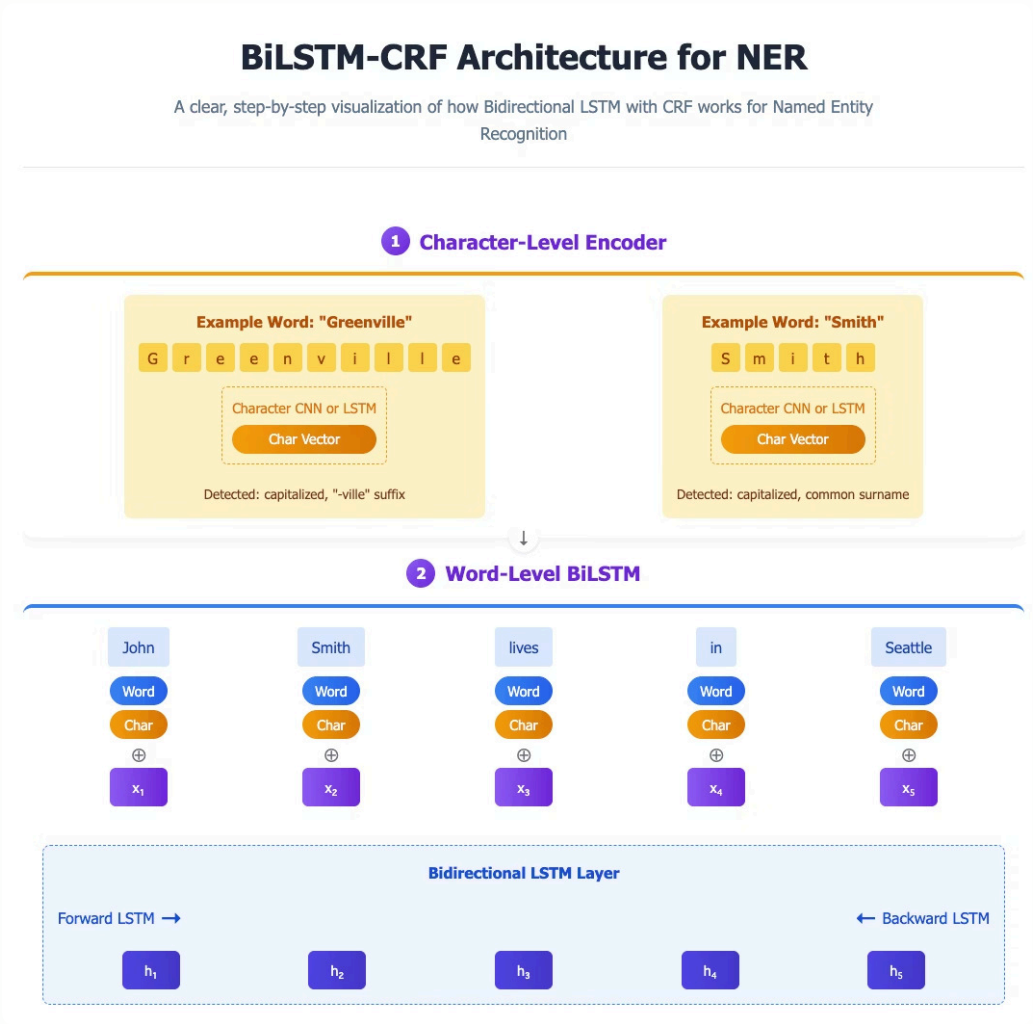
CRF Layer

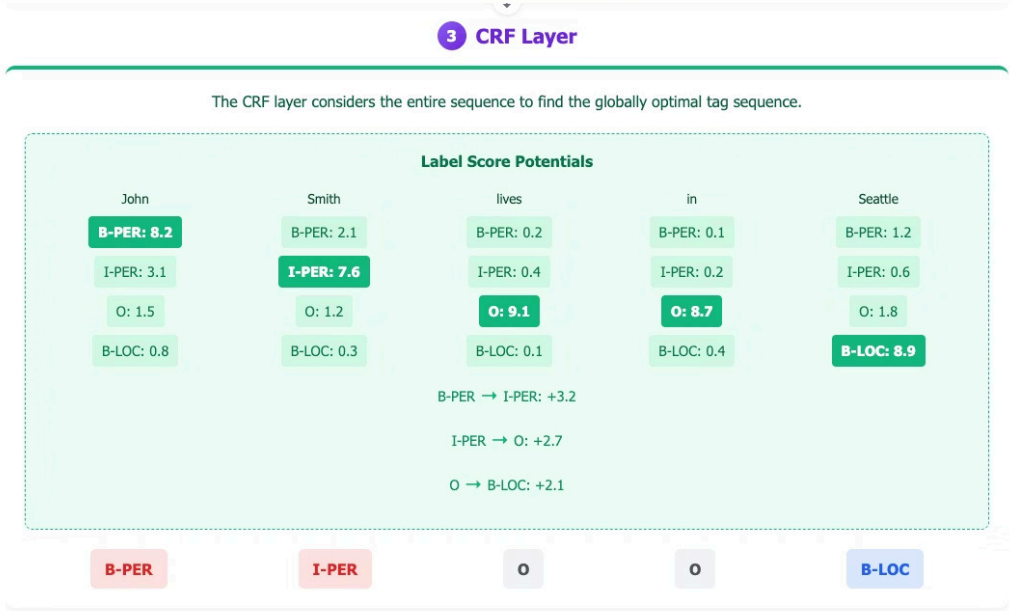
Adds transition scores and computes the best sequence of labels that maximizes the total score.

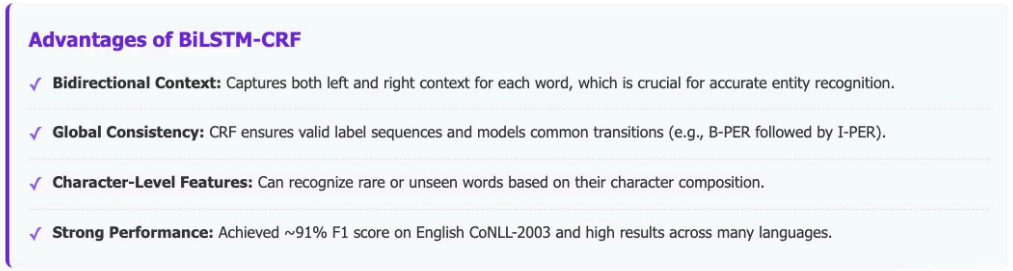
Dropout Regularization

Applied to embeddings and LSTM outputs to prevent overfitting during training.

BiLSTM-CRF Architecture







BiLSTM-CRF Architecture

1 Character-Level Encoder — Spelling Matters!

📌 What it does:

Looks at how the word is spelled, letter by letter.

🧠 Why?

Words like *"running"*, *"Runner"*, or *"Run"* are related — so the model learns patterns from **prefixes**, **suffixes**, and **capitalization**.

```
[0.25, -0.11, 0.82, ..., 0.37] → A small vector that encodes its spelling
```

🔤 Example:

- Input: "Google" → ['G', 'o', 'o', 'g', 'l', 'e']
- Output: A small vector that encodes its spelling.

✅ **Helps the model understand unknown or rare words**

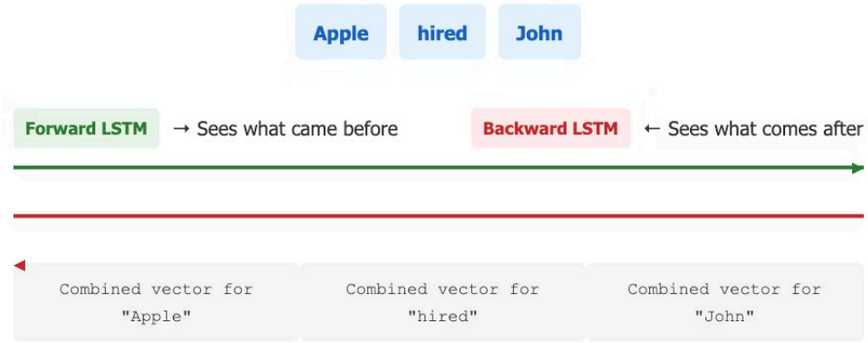
2 Word-Level BiLSTM — Context is Everything

📌 What it does:

Reads the sentence **left to right** and **right to left** to understand the context of each word.

🧠 Why?

The meaning of a word changes depending on its neighbors.



Bidirectional LSTM = Two brains:

- One sees the **past** (left side)
- One sees the **future** (right side)

📘 Example: In "Apple hired John", BiLSTM helps know:

- "Apple" is a company, not a fruit.
- "John" is likely a person because of "hired" before it.

✅ **Output: A context-rich vector for each word**

BiLSTM-CRF Architecture

3 CRF Layer — Sees the Big Picture

📌 **What it does:**

Decides the **best possible label sequence** for the sentence (e.g., B-PER, I-PER, O, B-ORG...).

🧩 **Why not just classify each word alone?**

Because labels depend on each other!

CRF learns these transition scores and uses them to ensure consistent labeling

💡 **Example:**

If "John" is tagged B-PER, then "Smith" should probably be I-PER, not O.

✅ **The CRF adds "transition scores" and picks the most consistent sequence of tags**

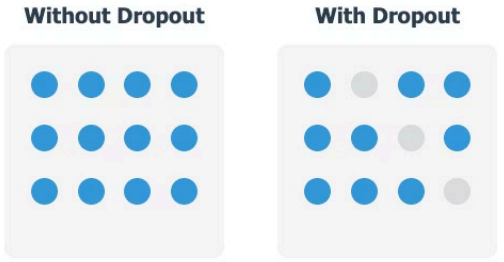
4 Dropout Regularization — Helps the Model Generalize

📌 **What it does:**

During training, it randomly drops some connections.

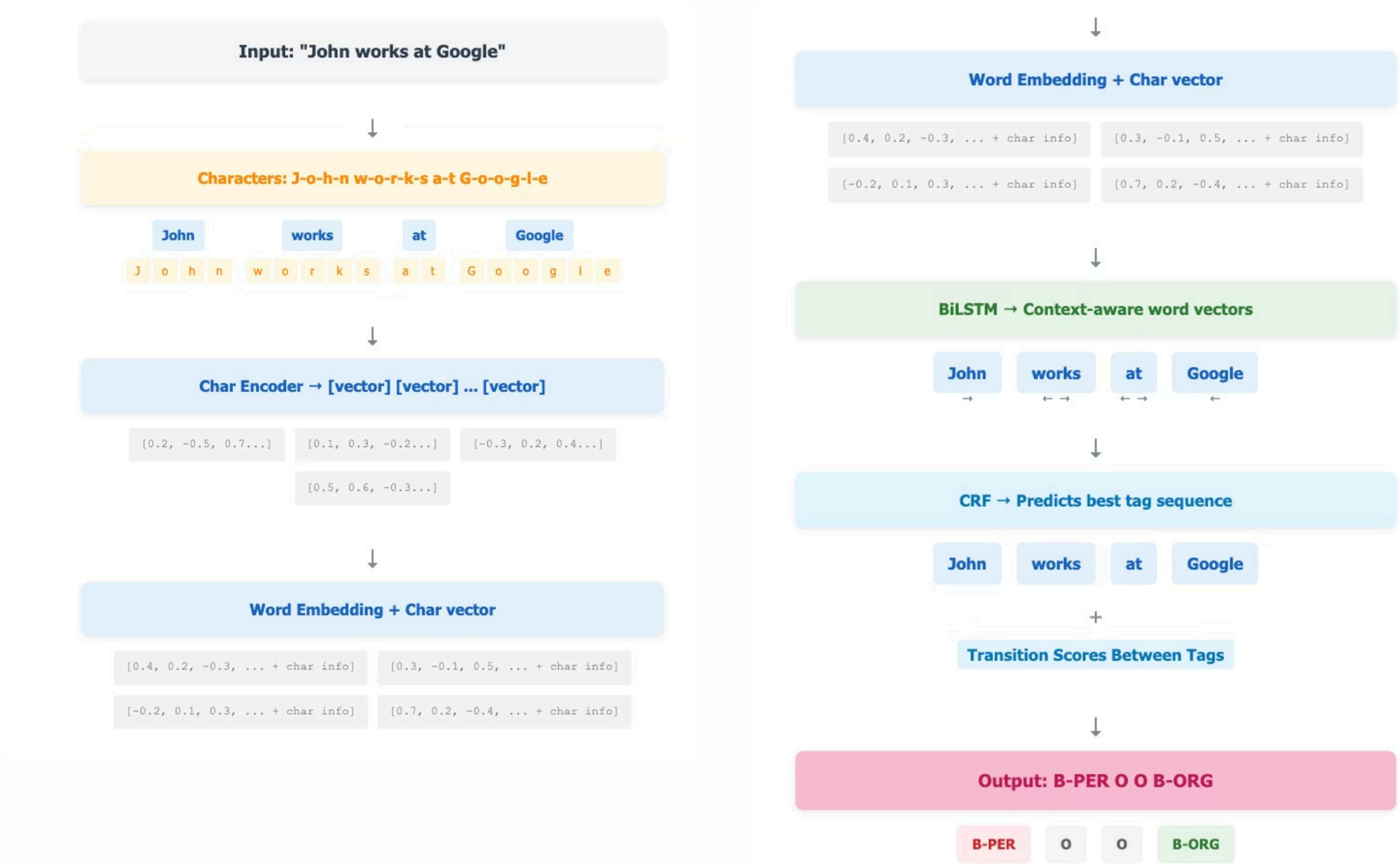
🧠 **Why?**

- Prevents the model from **memorizing** the training data.
- Forces it to **learn deeper patterns**.

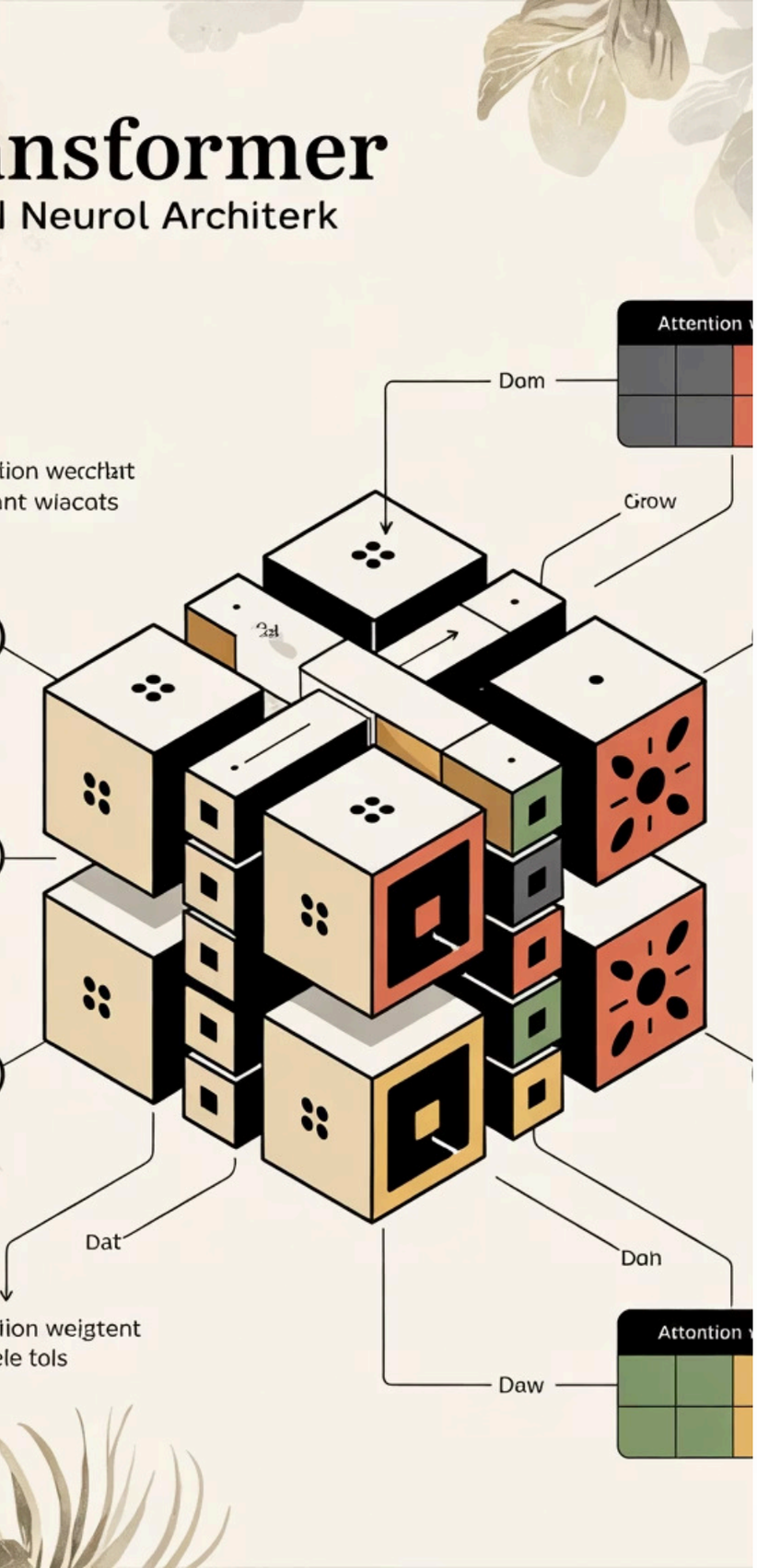


✅ **Makes the model more robust on unseen data**

BiLSTM-CRF Architecture Summary



BiLSTM-CRF Architecture Summary



Transformer-Based Models: The Current State-of-the-Art

Transformers have become the state-of-the-art method for NER. Unlike RNNs, Transformers use self-attention to handle contextual information and can be pre-trained on huge corpora.

BERT

Bidirectional Encoder Representations from Transformers, pre-trained on masked language modeling and next sentence prediction tasks.

RoBERTa

A robustly optimized BERT that improves training methodology and uses more data.

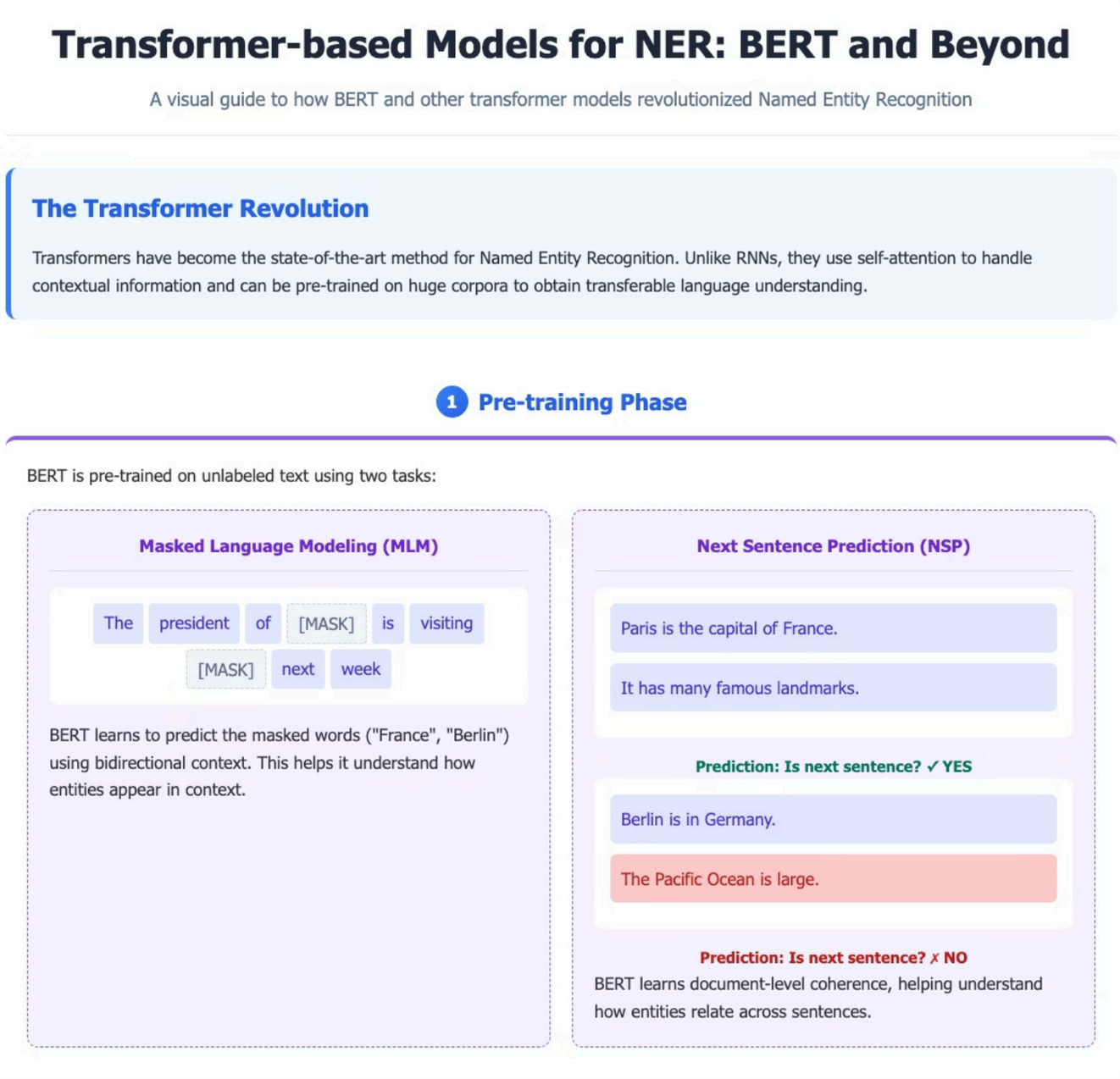
XLNet & ALBERT

Alternative transformer architectures with various trade-offs in performance and efficiency.

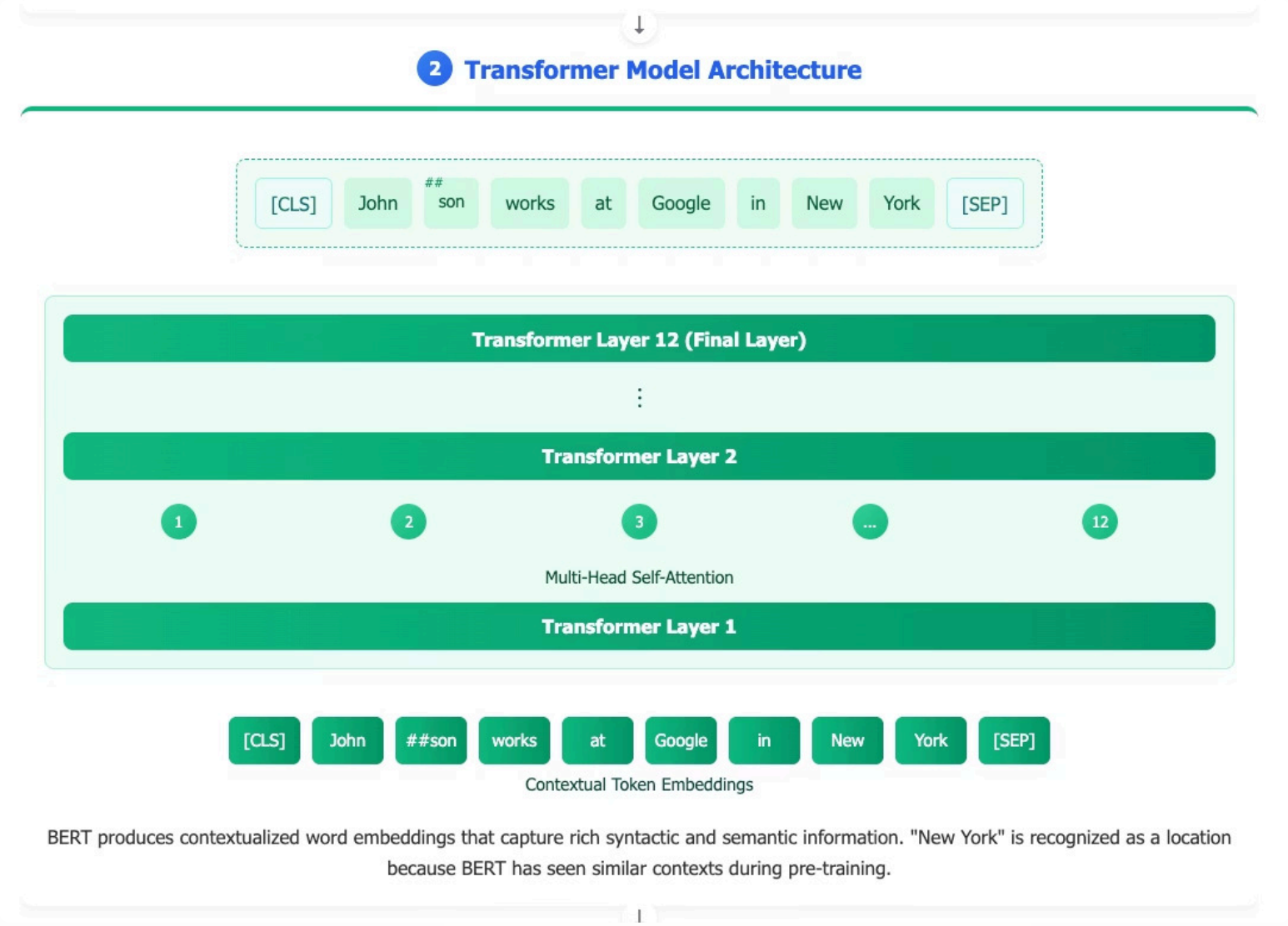
LayoutLM

Specialized for document processing, incorporating layout information useful for forms and structured documents.

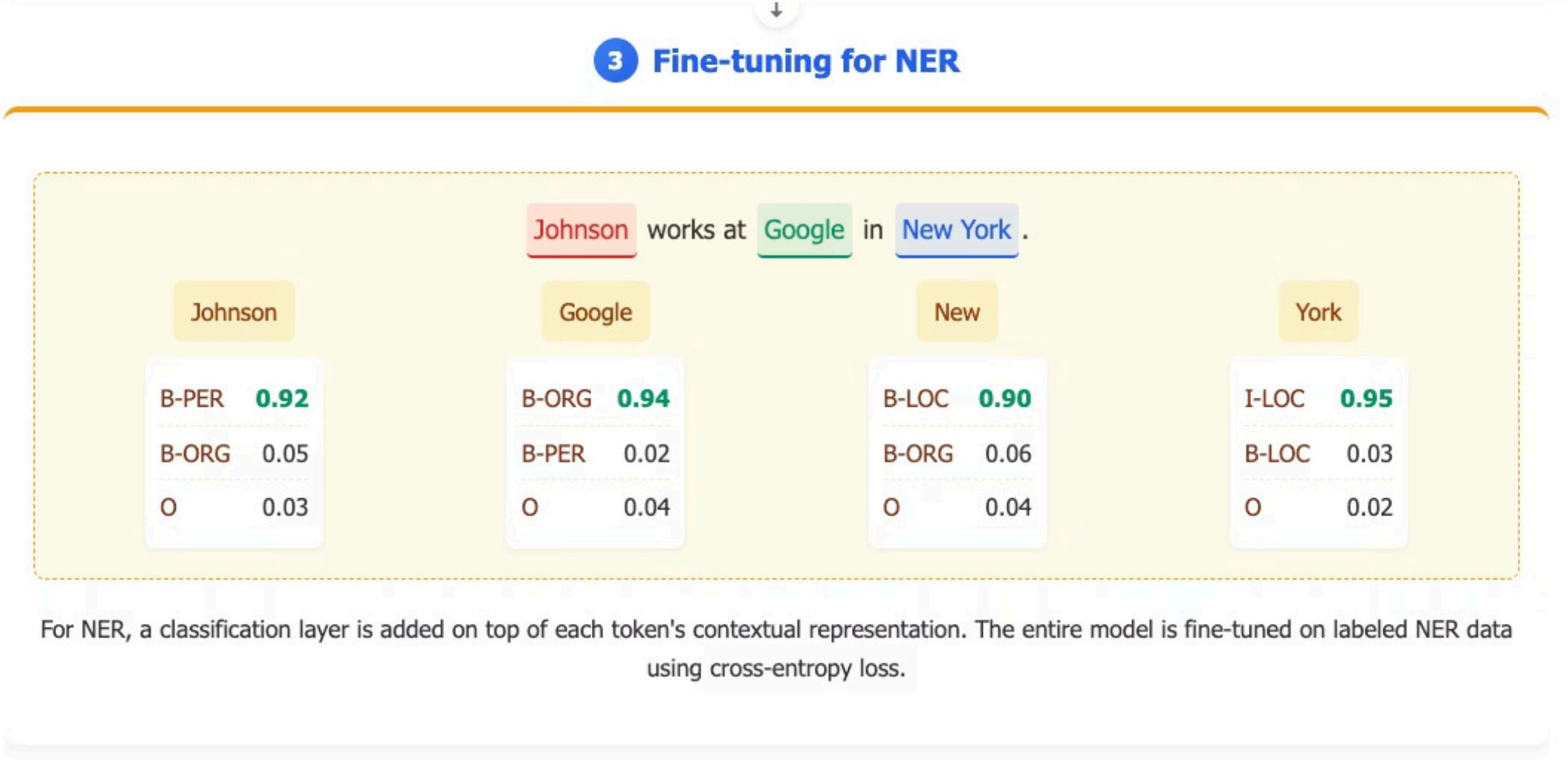
Transformer-based Models for NER



Transformer



Fine-tuning for NER



For NER, a classification layer is added on top of each token's contextual representation. The entire model is fine-tuned on labeled NER data using cross-entropy loss.

Transformer Models for NER

BERT

The original bidirectional transformer model that transformed NER performance.

- ✓ Pre-trained on massive text corpora using MLM and NSP
- ✓ Bidirectional context for each token
- ✓ Multilingual variants enable cross-lingual transfer

CoNLL-2003 (English): ~92.8 F1

RoBERTa

A robustly optimized version of BERT with improved training methodology.

🤖 Transformer-based Models for NER — Made Simple

Understanding how BERT and other transformers revolutionized Named Entity Recognition

🌟 Why Transformers?

Transformers **revolutionized NER** by doing one powerful thing: They **look at every word in a sentence at the same time** and understand how each word relates to all others — using something called **self-attention**.

John works at Google

Self-attention allows each word to "pay attention" to all other words, learning which ones are most important for understanding its meaning and role.

Unlike RNNs, which read from left to right:

- Transformers process the entire sentence in one go
- Each word can directly access information from every other word
- This captures long-range dependencies better than sequential models

How BERT Works in 3 Steps

1 Pretraining (Learning the Language)

BERT is trained like a detective solving word puzzles.

Masked Language Modeling (MLM)

The capital of France is **[MASK]**.

BERT predicts: "Paris"

Next Sentence Prediction (NSP)

[Sentence A] It's raining outside.
[Sentence B] I need an umbrella.

BERT predicts: "Are these sentences related? YES"

What does BERT learn?

- Grammar and syntax patterns
- Semantic relationships between words
- Common world knowledge
- Context-dependent word meanings

✅ **Result: BERT learns deep knowledge of how language works**

2 Fine-tuning for NER

Now we teach BERT a **specific skill**: labeling each word in a sentence (like B-PER, I-LOC, etc.).

Fine-tuning Process:

- Taking a **pretrained BERT model** (like bert-base-cased)
- **Adding a classification layer** on top of BERT (1 label per token)
- **Training it** on an annotated NER dataset

Example:

Training BERT to recognize that:

"John" is a person name

"Google" is an organization

"Paris" is a location

🎯 **Goal: For each word, BERT now outputs a label**

3 Why BERT is Great for NER

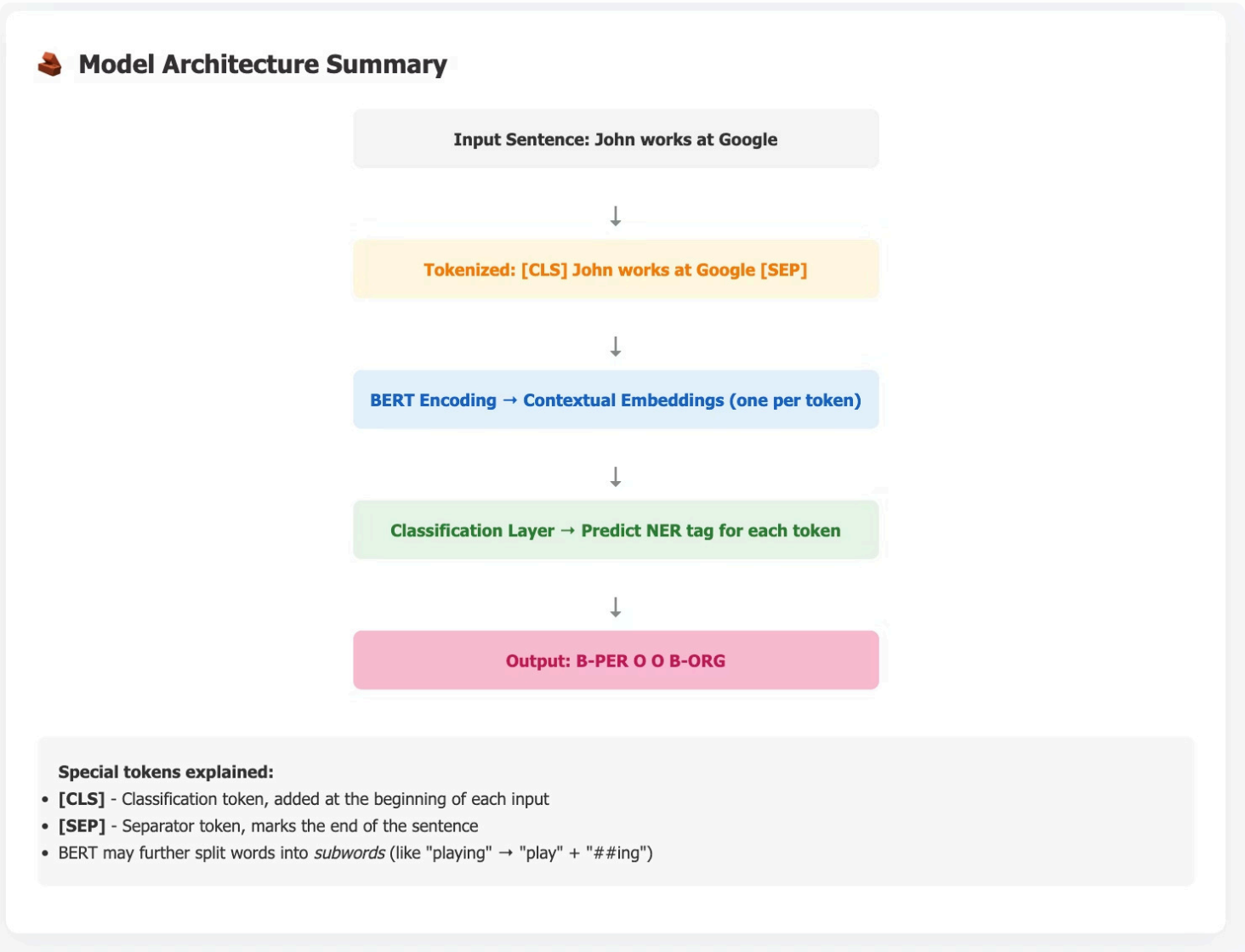
BERT understands:

- "Paris" after "in" = likely a **location**
- "Apple" before "hired" = likely a **company**

Advantages over previous approaches:

- It looks at the **whole sentence**, not just the previous word (unlike RNNs)
- It's been trained on **huge amounts of data**, so it brings in knowledge we didn't manually give it
- Captures **bidirectional context**, understanding both what came before and after
- Has strong **transfer learning** abilities - knowledge learned in pretraining helps with NER

Model Architecture Summary



✨ Add-ons & Variants

Variant	What It Improves
RoBERTa	Better training tricks for BERT (no NSP)
ALBERT	Lighter, faster version of BERT
XLNet	Predicts words in more flexible orders
LayoutLM	Includes layout info (great for documents/forms)

🛠️ Modern NER Techniques Built on Transformers

Approach	Idea
BERT + CRF	Use BERT to get token embeddings, then CRF to improve label consistency
Adapters	Small plug-in modules to fine-tune without changing the whole model
Gazetteer-aware NER	Feed in external lists like known cities, companies, etc.
Multi-task Learning	Train NER along with other tasks like POS tagging

🧰 Tools You Can Use

spaCy

Fast, production-ready NER

HuggingFace

Pretrained BERT models, easy fine-tuning

Flair

Combines BERT, GloVe, etc. for strong performance

AllenNLP

Easy experimentation with BiLSTM and transformers

Stanza

Covers many languages, from Stanford NLP group

🎓 Final Takeaway

Transformers like BERT **don't just read left to right** — they understand the **entire sentence all at once**, which makes them **incredibly powerful** for NER. Once fine-tuned, they achieve **state-of-the-art performance** with very little manual tweaking.

NER Implementation: Traditional Approaches

Let's examine how NER is implemented using traditional approaches like NLTK and CRF-based models.

NLTK Example

```
import nltk
nltk.download('maxent_ne_chunker')
nltk.download('words')
sentence = "Barack Obama was born in Hawaii."
tokens = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(tokens)
ne_tree = nltk.ne_chunk(pos_tags)
print(ne_tree)
```

Output

```
(S (PERSON Barack/NNP Obama/NNP)
   was/VBD born/VBN in/IN
   (GPE Hawaii/NNP) ./.)
```

NLTK correctly identifies "Barack Obama" as a PERSON entity and "Hawaii" as a GPE (geo-political entity).

Modern NER Implementation: spaCy and Transformers

Modern NER implementations leverage pre-trained models and simple APIs to achieve state-of-the-art results with minimal code.

spaCy Example

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple Inc. is looking at buying U.K. startup for
$1 billion")
for ent in doc.ents:
    print(ent.text, ent.label_)
```

Output:

```
Apple Inc. ORG
U.K. GPE
$1 billion MONEY
```

Hugging Face Transformers

```
from transformers import pipeline
nlp = pipeline("ner",
       model="dslim/bert-base-NER",
       grouped_entities=True)
results = nlp("Barack Obama was born in Hawaii.")
for res in results:
    print(res)
```

Output:

```
{'entity_group': 'PER',
 'score': 0.9995,
 'word': 'Barack Obama'}
{'entity_group': 'LOC',
 'score': 0.9988,
 'word': 'Hawaii'}
```

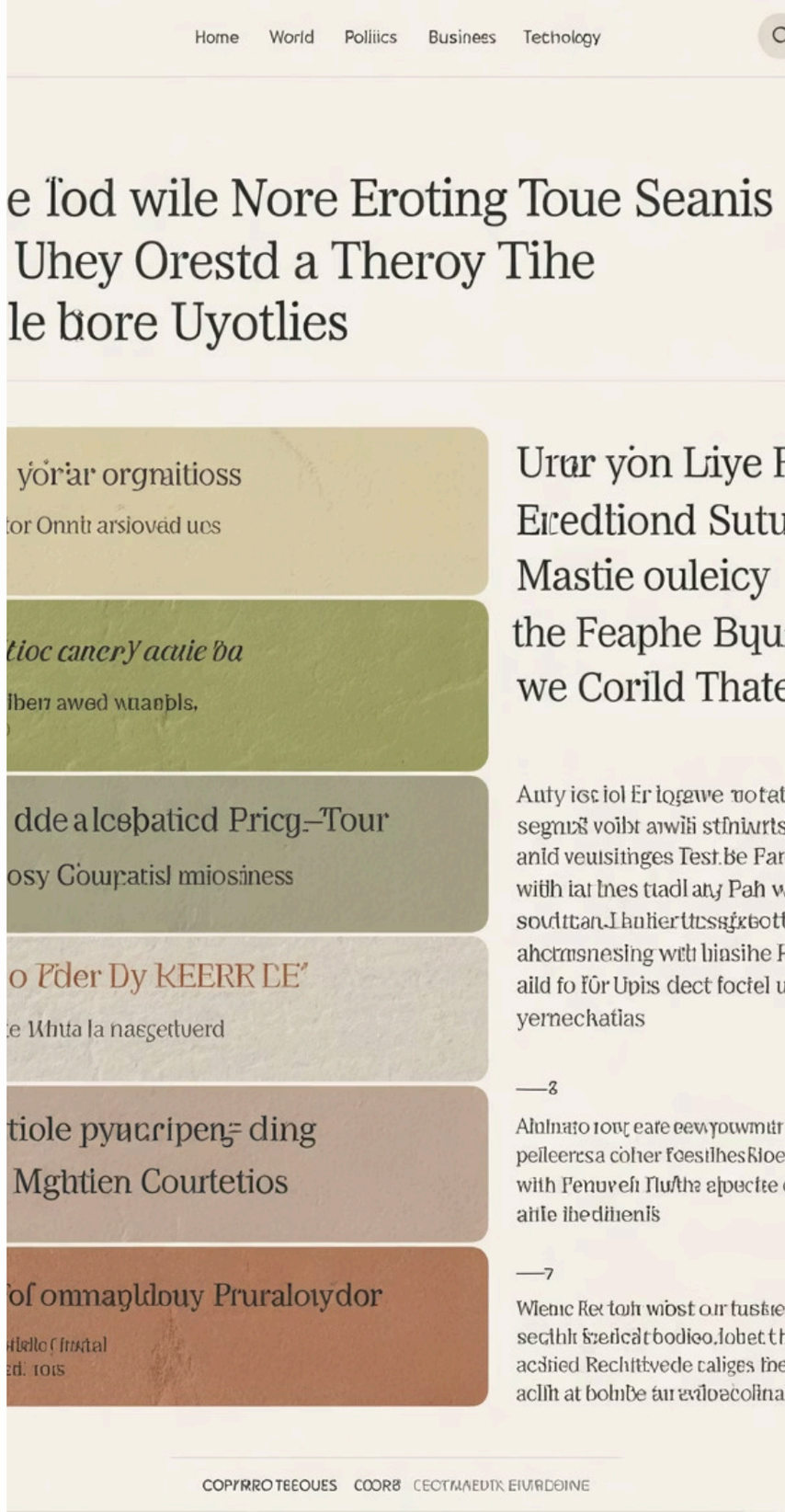
Real-World Example: News Text Analysis

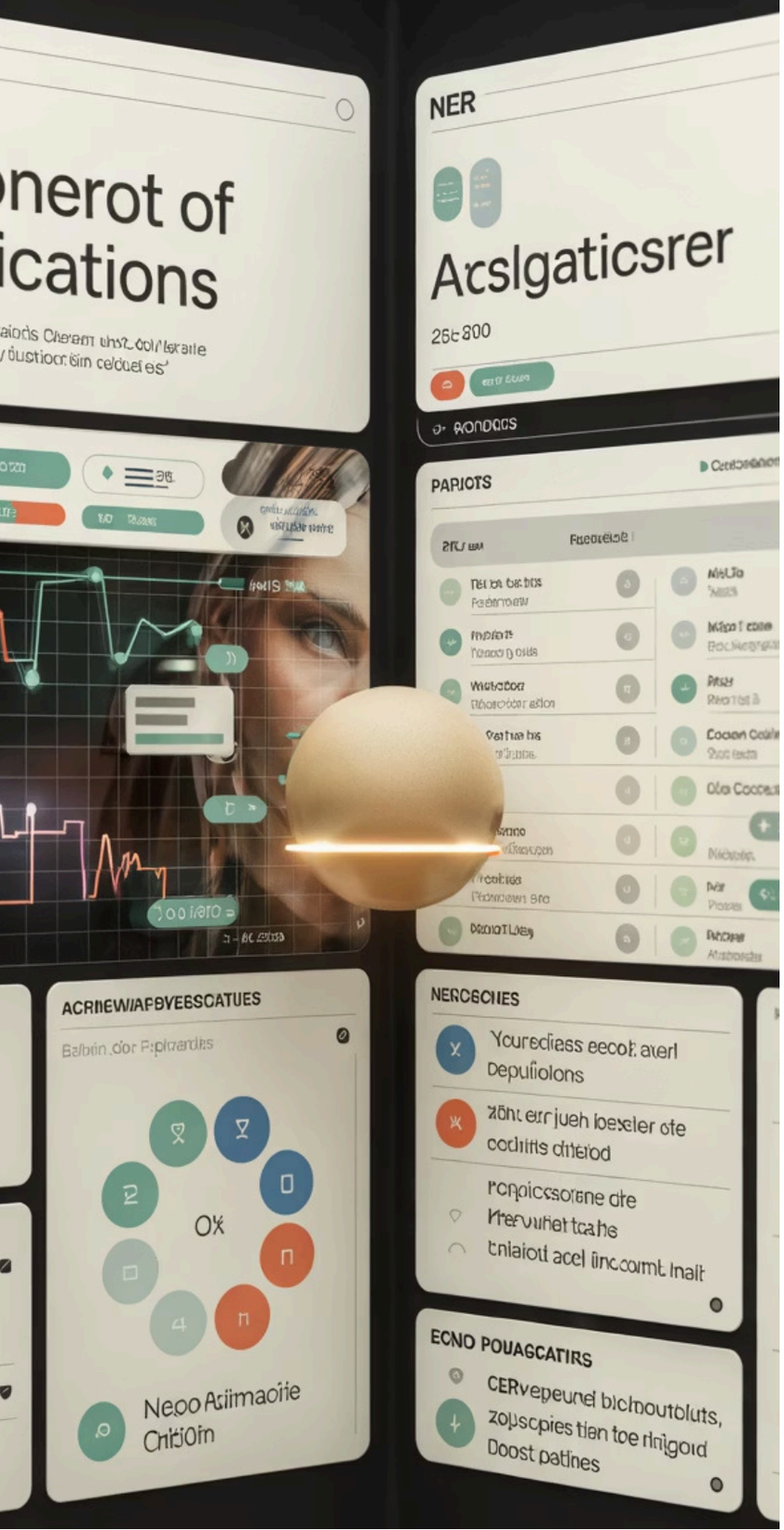
Let's see how NER works on a short news excerpt:

> "Google LLC announced a partnership with Pfizer Inc. on Monday. The California-based tech giant and the New York pharmaceutical company
> will collaborate on new healthcare initiatives."

Entity	Type
Google LLC	ORG
Pfizer Inc.	ORG
Monday	DATE
California	GPE
New York	GPE

A good NER system captures all these different entity types, which can then be used for further processing like knowledge base linking or article categorization.





Domain-Specific NER Applications

Biomedical NER

Identifies genes, proteins, diseases, chemicals, and drugs in scientific literature and clinical notes. Used for database curation and patient data anonymization.

Financial NER

Extracts company names, stock tickers, monetary amounts, and percentages from financial reports and news for automated analysis.

Legal NER

Targets case names, law provisions, court names, and legal citations in documents to aid in hyperlinking references and summarization.

Social Media NER

Handles noisy text with creative spelling and hashtags to identify trends and entities mentioned in platforms like Twitter.

Named Entity Recognition Across Domains

NER is applied in many domains, each with unique entity categories and challenges. Here's a look at how NER is used across different specialized fields.

Biomedical NER

Extraction of entities from research articles and clinical notes.

GENE DISEASE PROTEIN DRUG

BRCA1 mutations are linked to breast cancer .

- Complex vocabularies (gene names can be alphanumeric)
- Requires domain knowledge (e.g., "TNF-alpha" is a protein)

$ Financial NER

Extraction of financial entities from reports and news.

COMPANY TICKER DATE PERCENT

Tesla Inc. (TSLA) reported a 50% increase in revenue in Q4 2021 .

- Benefits from custom lexicons (company names, stock tickers)
- Requires disambiguation (Apple company vs apple fruit)

Legal NER

Identification of legal entities in documents and case law.

CASE COURT LAW JUDGE

In Roe v. Wade , the Supreme Court of the United States held that...

- Formal language with specific structures
- Long named entities (titles of laws)

Social Media NER

Entity extraction from noisy social media text.

PERSON PRODUCT LOCATION EVENT

Just watched Game of Thrones with @JohnDoe in Central Park ! #GoT

- Noisy text (lack of capitalization, creative spelling)
- New entity types (TV shows, sports teams, hashtags)

Geospatial NER

Identification of location entities for mapping and navigation.

FACILITY ADDRESS CITY COUNTRY

Meet me at the Eiffel Tower in Paris , France .

- Integration with geodatabases for validation
- Handling of ambiguous place names

Multilingual NER

Adapting NER techniques to different languages and scripts.

STANDARD SCRIPT MORPHOLOGY

Languages like Arabic, Chinese, and Japanese lack capitalization cues for entity recognition.

- Complex morphology in many languages
- Lack of capitalization cues in some scripts
- Word segmentation issues (e.g., Chinese)

Challenges in Named Entity Recognition

Nested Entities

Entities that overlap or nest within each other, like "Bank of America" (Organization) inside "Bank of America Tower" (Location).

Ambiguity

"Washington" could be a person, location, or organization depending on context, requiring sophisticated disambiguation.

Domain Adaptation

Models trained on news may fail on biomedical text due to vocabulary and style differences.

Multilingual Issues

Languages like Chinese or Arabic lack capitalization cues, while German has compound words that complicate entity boundaries.

NER Benchmarks and Datasets

A comprehensive overview of the key datasets that have shaped Named Entity Recognition research and evaluation

MUC-6 & MUC-7

1995-1997

Message Understanding Conference datasets that established the foundation for NER evaluation. These early datasets focused on news articles and defined core evaluation metrics still used today.

DOMAIN **News Articles**	LANGUAGE **English**
FORMAT **SGML**	SOURCE **Wall Street Journal**

- ENAMEX (PER, ORG, LOC)
- TIMEX (time expressions)
- NUMEX (numerical)

CoNLL-2002/2003

2002-2003

The most widely used benchmarks for NER evaluation. These datasets are derived from Reuters news corpus and have become the de facto standard for comparing NER systems.

TRAIN SIZE **~204K tokens (EN)**	TEST SIZE **~51K tokens (EN)**
LANGUAGES **EN, DE, ES, NL**	SOTA F1 **88-93%**

- PER (Person)
- ORG (Organization)
- LOC (Location)
- MISC (Miscellaneous)

ACE

2003-2005

Automatic Content Extraction datasets include nested entity annotations and fine-grained subtypes, allowing for more complex entity recognition tasks and evaluations.

FEATURE **Nested Entities**	SOURCES **News, Broadcast**
LANGUAGES **EN, AR, CN**	TASK FOCUS **Entity Relations**

- Fine subtypes (FAC, WEA)
- Nested annotations
- Entity relations

OntoNotes 5.0

2011-2012

Large corpus with text from diverse sources including news, conversations, web content, with multiple annotation layers beyond just named entities.

SIZE **2.9M words**	LANGUAGES **EN, CN, AR**
GENRES **6 (News, Web, etc.)**	ENTITY TYPES **18 fine-grained**

- PERSON
- ORG
- GPE
- NORP
- PRODUCT
- EVENT

WNUT Emerging Entities

2017

Targets NER in noisy user-generated text from social media with a focus on emerging entity types not present in training data, testing generalization capabilities.

DOMAIN **Social Media**	SOURCE **Twitter, Reddit, etc.**
CHALLENGE **Noisy Text**	FOCUS **Emerging Entities**

- Person
- Location
- Corporation
- Product
- Creative-work
- Group

Biomedical Datasets

Various Years

Specialized datasets for biomedical and clinical text processing, focusing on domain-specific entities such as genes, proteins, diseases, and chemicals.

DOMAIN **Biomedical**	SOURCES **Research Papers, Clinical Notes**
KEY DATASETS **GENIA, BC5CDR, i2b2**	USE CASE **BioNER**

- Gene/Protein
- Disease
- Chemical
- Species
- PHI (Protected Health Info)

NER Evaluation Metrics

Precision

Measures how many of the identified entities are correct

```
P = TP / (TP + FP)
```

Recall

Measures how many of all actual entities were correctly identified

```
R = TP / (TP + FN)
```

Exact Match

Entity is correct only if both span boundaries and type match exactly with gold standard

F1 Score

Harmonic mean of precision and recall, balancing both measures

```
F1 = 2 × (P × R) / (P + R)
```

Dataset Accessibility

LDC

Linguistic Data Consortium hosts ACE, OntoNotes, and some CoNLL datasets

Hugging Face

Datasets library provides easy access to CoNLL, WikiAnn and other NER datasets

spaCy

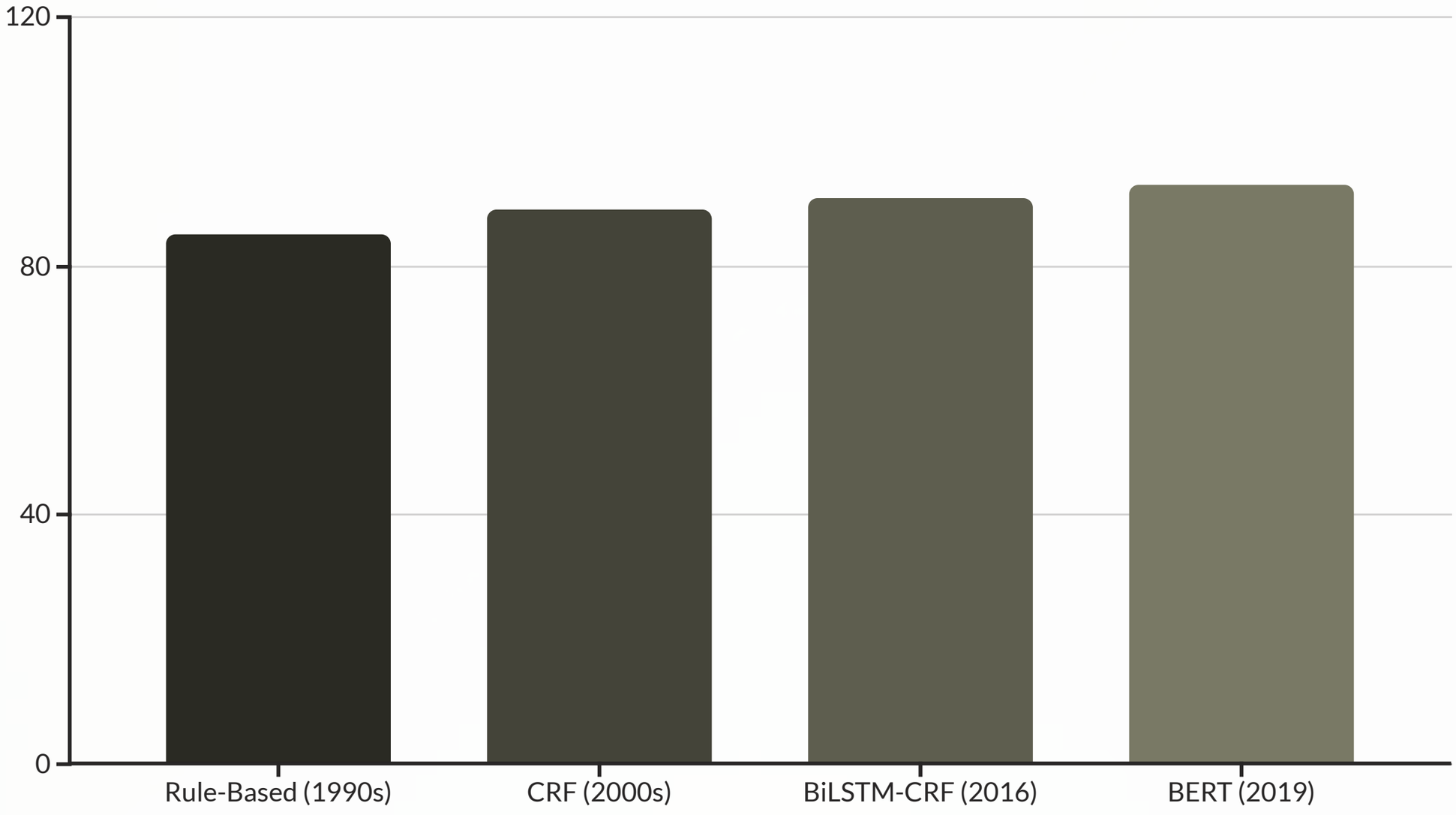
Provides sample of OntoNotes for English in its models for experimentation

GitHub

Many datasets are available through GitHub repositories with annotations

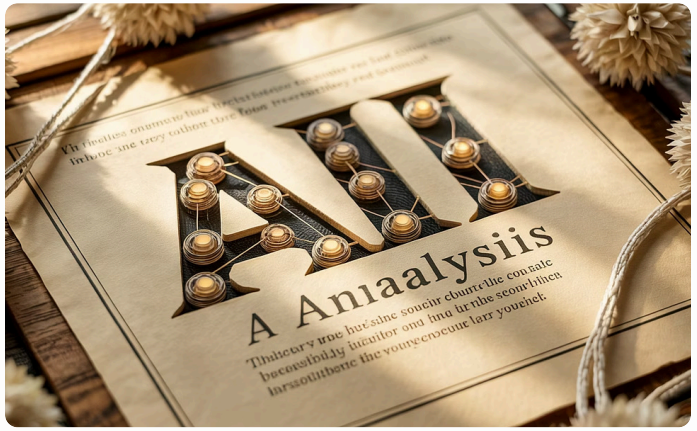
Evaluation Metrics and Benchmarks

NER systems are typically evaluated using precision, recall, and F1-score on entity spans. The most common evaluation uses strict (exact) matching, where an entity is counted as correct only if both the span and type match exactly.



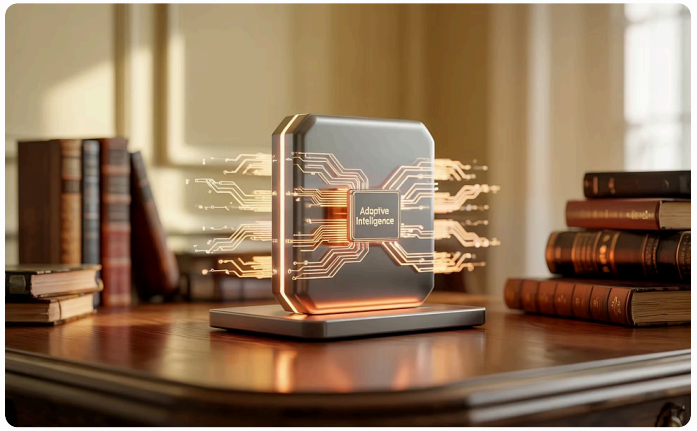
Key benchmarks include CoNLL-2003 (English, German), OntoNotes 5.0 (18 fine-grained types), and domain-specific datasets like GENIA for biomedical text.

Future Directions in NER Research



Nested Entity Recognition

Developing models that can handle overlapping entities, using techniques like hypergraph representations or layered tagging approaches.



Few-Shot Learning

Creating models that can learn new entity types from just a handful of examples, using meta-learning or prompt-based approaches.

Cross-Lingual Transfer

Improving zero-shot capabilities across languages, allowing models trained on resource-rich languages to perform well on low-resource languages.

Conclusion: The Impact and Future of NER

Named Entity Recognition has evolved from simple pattern-matching to sophisticated neural models that achieve over 90% F1 on benchmark datasets. This evolution reflects broader trends in NLP: from manual rule engineering to statistical learning to deep neural networks with minimal feature engineering.

30+	90%+	18+
Years of Development	Accuracy	Entity Types
From MUC-6 in 1995 to today's transformer models	F1 scores on standard benchmarks	In modern fine-grained systems

As NER technology continues to improve, we can expect more integration with knowledge bases, better generalization to unseen entity types, and improved contextual understanding. NER will remain a foundational task wherever text needs to be structured, making it a critical tool in the NLP toolbox.

Appendices

Hidden Markov Models (HMM) for NER

A probabilistic sequence model that treats NER as a sequence labeling problem

What are Hidden Markov Models?

Definition

A Hidden Markov Model (HMM) is a statistical model that represents a sequence of observations as being generated by transitions between "hidden" states. In the context of Named Entity Recognition, the observations are words in a text, and the hidden states are entity tags (PERSON, LOCATION, ORGANIZATION, etc.).

HMMs are based on the Markov assumption that the current state depends only on the previous state, not on the entire history of states.

Hidden Markov Models for Named Entity Recognition

Applying Probabilistic Sequence Labeling to Extract Named Entities from Text

1 HMMs in Named Entity Recognition

In Named Entity Recognition (NER), Hidden Markov Models (HMMs) are used to identify and classify named entities (like persons, organizations, and locations) in text. HMMs are well-suited for NER because they can model the sequential nature of language.

Key Insight:

In NER using HMMs, the **hidden states** are the entity labels (e.g., B-PER, I-ORG, O), and the **observations** are the words in the text. Our goal is to find the most likely sequence of entity labels given the words we observe.

HMM Components

States

In NER, states typically use the BIO (Beginning-Inside-Outside) tagging scheme:

- **B-X**: Beginning of entity type X
- **I-X**: Inside (continuation) of entity type X
- **O**: Outside any entity (not part of a named entity)

Example state set: {O, B-PER, I-PER, B-LOC, I-LOC, B-ORG, I-ORG}

Observations

The observed sequence consists of words from the text.
Example: ["John", "Smith", "visited", "New", "York", "yesterday"]
Each observation is associated with a "hidden" state that we want to discover.

Transition Probabilities

The probability of moving from one state to another.
Example: P(B-PER → I-PER) = 0.8
This captures the likelihood that after the beginning of a person entity, the next token is still part of the same person entity.

Emission Probabilities

The probability of observing a particular word given a specific state.
Example: P("John" | B-PER) = 0.1
This captures how likely the word "John" is to appear at the beginning of a person entity.

HMM Mathematical Formulation

Mathematical Formulation

HMM Components

A Hidden Markov Model is formally defined by:

- **Q**: Set of states (e.g., {O, B-PER, I-PER, B-LOC, I-LOC})
- **O**: Set of observations (words in vocabulary)
- **A**: Transition probability matrix
- **B**: Emission probability matrix
- **π**: Initial state distribution

$$P(X, Y) = P(y_1) \prod_{t=1}^{T} P(y_t \mid y_{t-1}) \times P(x_t \mid y_t)$$

Joint Probability Explanation

This equation calculates the joint probability of an observation sequence $X = (x_1, x_2, ..., x_T)$ and a state sequence $Y = (y_1, y_2, ..., y_T)$ where:

- **$P(y_1)$**: The probability of starting in state y_1 (Initial state probability)
- **$P(y_t \mid y_{t-1})$**: The probability of transitioning from state y_{t-1} to state y_t (Transition probability)
- **$P(x_t \mid y_t)$**: The probability of observing x_t in state y_t (Emission probability)

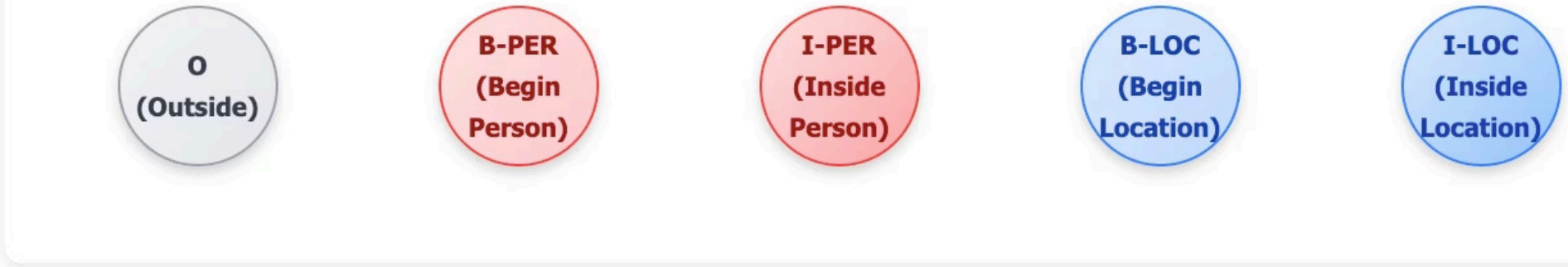
For NER, this helps us find the most likely sequence of entity tags (states) given a sequence of words (observations).

HMM for NER: A Worked Example

HMM for NER: A Worked Example

Input Sentence: "John visited New York yesterday"

Simplified HMM State Diagram



Simplified state diagram showing possible transitions between entity states

Sample Probability Tables

Transition Probabilities:

From / To	O	B-PER	I-PER	B-LOC	I-LOC
O	0.6	0.2	0.0	0.2	0.0
B-PER	0.2	0.0	0.8	0.0	0.0
I-PER	0.7	0.1	0.2	0.0	0.0
B-LOC	0.2	0.0	0.0	0.0	0.8
I-LOC	0.7	0.0	0.0	0.1	0.2

Sample Probability Tables

Emission Probabilities (sample):

State / Word	John	visited	New	York	yesterday
O	0.001	0.05	0.001	0.001	0.03
B-PER	0.1	0.001	0.005	0.005	0.001
I-PER	0.01	0.001	0.001	0.01	0.001
B-LOC	0.001	0.001	0.08	0.01	0.001
I-LOC	0.001	0.001	0.01	0.09	0.001

Decoding: Finding the Most Likely State Sequence

Decoding: Finding the Most Likely State Sequence

To find the most likely sequence of entity tags, we use the Viterbi algorithm:

1 Initialization

Initialize the first column of the Viterbi matrix using initial probabilities and first word emissions.

2 Recursion

For each time step t and each state j, calculate:
viterbi[t][j] = $\max_i$(viterbi[t-1][i] × transition[i][j] × emission[j][observation[t]])

3 Backtracking

Trace back through the matrix to find the most likely state sequence.

Simplified Viterbi Table for Our Example

This is a simplified representation of the Viterbi algorithm applied to our example sentence.

State / Word	John	visited	New	York	yesterday
O	0.0006	**0.0015**	0.00015	0.00001	**0.00003**

Practical Example: Viterbi Algorithm for NER

6 Practical Example: Viterbi Algorithm for NER

The Viterbi algorithm efficiently finds the most likely sequence of entity tags by keeping track of the best path to each state at each time step.

Viterbi Decoding for "John works at Google"

John	works	at	Google

Step-by-Step Viterbi Process:

1. **Initialization:** For each possible entity tag s, compute Viterbi variable $v_1(s) = \pi_s \cdot b_s(\text{John})$
2. **Recursion:** For each word position t = 2 to 4 and each possible entity tag s:

$$v_t(s) = \max_{s'} [v_{t-1}(s') \cdot a_{s',s} \cdot b_s(x_t)]$$

   Where s' represents the previous entity tag, and we keep track of which s' gave the maximum.
3. **Termination:** The probability of the best path is $\max_s [v_4(s)]$
4. **Backtracking:** Trace back the sequence of states that led to the maximum probability.

↓

B-PER	O	O	B-ORG

HMM Components for NER

3 HMM Components for NER

An HMM for NER is defined by these key components:

States (S)

The set of possible entity tags: {B-PER, I-PER, B-ORG, I-ORG, B-LOC, I-LOC, O, ...}

Observations (O)

The vocabulary of words that appear in the text

Initial Probabilities (π)

The probability of a sentence starting with each entity tag

Transition Probabilities (A)

The probability of transitioning from one entity tag to another

	B-PER	I-PER	O	...
B-PER	0.01	0.85	0.10	...
I-PER	0.01	0.70	0.25	...
O	0.15	0.01	0.60	...

Emission Probabilities (B)

The probability of observing a particular word given an entity tag

	Barack	visited	Microsoft	...
B-PER	0.02	0.001	0.001	...
I-PER	0.003	0.001	0.001	...
B-ORG	0.001	0.001	0.05	...

Key Observation:

The transition probabilities capture linguistic patterns like "after a B-PER tag, an I-PER tag is likely to follow" (e.g., in "Barack Obama"). The emission probabilities capture which words are likely to be of which entity type.